



Parul University

FACULTY OF ENGINEERING AND TECHNOLOGY
BACHELOR OF TECHNOLOGY

Image Processing

(303105382)

7th SEMESTER

COMPUTER SCIENCE ENGINEERING DEPARTMENT

Laboratory Manual



**Image Processing PRACTICAL BOOK ELECTRONICS AND
COMMUNICATION ENGINEERING DEPARTMENT**

PREFACE

It gives us immense pleasure to present the first edition of **Image Processing** Practical Book for the B. Tech., 4th year students of **PARUL UNIVERSITY**.

The **Image Processing** theory and laboratory course at **PARUL UNIVERSITY, WAGHODIA, VADODARA** is designed in such a way that students will have basic understanding of the subject in the theory classes and then they have to try their hands in the experiments session to realize problems learnt during the theoretical sessions. The main objective of the **Image Processing** laboratory course is: Learning **Image Processing** through Experimentations. All the experiments are designed to understand and explore **Image Processing** in order to render the student's various approaches used to processing and improving the quality of the images. Further, it will give exposure to students when to use the specific algorithms for a given applications.

The objective of **Image Processing** Practical Book is to provide a comprehensive source for all the experiments included in the **Image Processing** laboratory course. It also explains all the aspects related to every experiment such as: understanding of basics of algorithms, how to write program in MATLAB for image processing in order to make them understand the fundamentals of Image Processing.

We acknowledge the authors and publishers of all the books which we have consulted while developing this Practical book. Hopefully this *Image Processing Practical Book* will serve the purpose for which it has been developed.



INSTRUCTIONS TO STUDENTS

1. Be prompt in arriving to the laboratory and always come well prepared for the experiment.
2. Be careful while working on the equipment operated with high voltage power supply.
3. Work quietly and carefully. Give equal opportunity to all your fellow students to work on the instruments.
4. Every student should have his/her individual copy of the **Image Processing** Practical Book.
5. Every student has to prepare the notebooks specifically reserved for the **Image Processing** Practical work — **Image Processing** Practical Bookl.
6. Every student has to necessarily bring his/her **Image Processing** Practical Book, Practical record Notebook, when he/she comes to the practical to perform the experiment.
7. All the algorithms/ codes/ observations have to be neatly recorded in the **Image Processing** Practical Class Notebook (as explained in the **Image Processing** Practical Book) and get it verified by the instructor before leaving the laboratory.
8. Do not share your programs with your fellow student. Every student has to render his/her own set of results by performing the experiment separately.
9. Do not forget to get the information of your next allotment (the experiment which is to be performed by you in the next laboratory session) before leaving the laboratory from the Technical Assistant.
10. The grades for the **Image Processing** Practical course work will be awarded based on your performance in the laboratory, regularity, recording of experiments in the **Image Processing** Practical Final Notebook, lab quiz, regular viva-voce and end-term examination.



CERTIFICATE

*This is to certify that Mr./Ms with
enrolment no. has successfully completed
his/her laboratory experiments in the **Image Processing** (203105471) from the
department of during
the academic year*



Date of Submission:.....

Staff In charge:.....

Head of Department:.....



TABLE OF CONTENT

Sr. No	Experiment Title	Page No		Date of Start	Date of Completion	Sign	Marks (out of 10)
		From	To				
1	To study and implement basic commands of MATLAB required for digital image processing technique, and various image file formats.						
2	To study the effect of down sampling and quantization techniques on the grayscale image.						
3	Write MATLAB code to enhance the visual quality of the image using point processing techniques, namely a) Image negative, b) Logarithmic transformation, and c) Power law transformation and gray level slicing technique.						
4	Write MATLAB code to display the histogram of the image and enhance the visual quality of the image with the help of histogram equalization technique.						
5	Write MATLAB code to perform the bit-plane slicing method on grayscale image.						
6	Write MATLAB code to demonstrate that the convolution in spatial domain is equivalent to multiplication in the frequency domain.						
7	Write MATLAB code to restore grayscale image from noisy image with the help of image restoration techniques.						
8	Write MATLAB code to perform Min, Median & Max Filtering on grayscale image.						
9	Write MATLAB code to blur the image using Ideal low pass, Butterworth low pass and Gaussian Low pass filter.						



PARUL UNIVERSITY
FACULTY OF ENGINEERING & TECHNOLOGY
Image Processing (303105382) B. Tech. 4th Year

10	Write MATLAB code to blur the image using Ideal High pass, Butterworth High pass and Gaussian High pass filter.						
----	---	--	--	--	--	--	--



EXPERIMENT NO: 1

AIM: To study and implement basic commands of MATLAB required for image processing technique, and various image file formats.

Software/Hardware:

Objective: The objective of the experiment is to,

- Get acquainted with Image processing commands available in the MATLAB.
- How to use a given command with the help of MATLAB Product Help.

Syntax: Syntax to be covered in this experiment,

1. **clc**

Clear Command Window

Syntax

clc

Description

clc clears all input and output from the Command Window display, giving you a "clean screen". After using clc, you cannot use the scroll bar to see the history of functions, but you still can use the up arrow to recall statements from the command history.

2. **Clear all**

Remove items from workspace, freeing up system memory

Syntax

clear

clear name

clear name1 name2 name3 ...

clear global name

clear -regexp expr1 expr2 ...

clear global -regexp expr1 expr2 ...

clear keyword

clear('name1','name2','name3',...)

Description

clear removes all variables from the workspace, releasing them from system memory.

Clear name removes just the M-file or MEX-file function or variable name from your base workspace. If called from a function, clear name removes name from both the function workspace and in your base workspace. You can use wildcards (*) to remove items selectively. For example, clear my* removes any variables whose names begin with the



string my. Clearing removes debugging breakpoints in M-files and reinitializes persistent variables. If name is global, clear removes it from the current workspace, but it remains accessible to any functions declaring it global. If name has been locked by mlock, it remains in memory.

Use a partial path to distinguish between different overloaded versions of a function. For example, clear polynom/display clears only the display method for polynom objects, leaving any other implementations in memory.

clear name1 name2 name3 ... removes name1, name2, and name3 from the workspace.

clear global name removes the global variable name. If name is global, clear name removes name from the current workspace, but leaves it accessible to any functions declaring it as global. Use clear global name to remove a global variable completely.

clear -regexp expr1 expr2 ... clears all variables that match any of the regular expressions listed. This option only clears variables.

clear global -regexp expr1 expr2 ... clears all global variables that match any of the regular expressions listed.

clear keyword clears the items indicated by keyword. See the following section, Inputs, for a list of keywords and their descriptions.

clear('name1','name2','name3',...) is the function form of the syntax. Use this form for variable names and function names stored in strings.

If you name a variable all, classes, functions, java, import, or variables, calling clear followed by that name deletes the variable with that name. clear does not interpret the name as a keyword in this context. For example, if the workspace contains variables a, all, b, and ball, clear all deletes the variable all only.

You can clear the handle of a figure or other object, but that does not remove the object itself. Use delete to remove objects and files. Deleting an object does not delete the variable, if any, used for storing its handle.

3. Close all

Remove specified figure

Syntax

close

close(h)

close name

close all

close all hidden



close all force
status = close(...)

Description

close deletes the current figure or the specified figure(s). It optionally returns the status of the close operation.

close deletes the current figure (equivalent to close(gcf)).

close(h) deletes the figure identified by h. If h is a vector or matrix, close deletes all figures identified by h.

close name deletes the figure with the specified name.

close all deletes all figures whose handles are not hidden.

close all hidden deletes all figures including those with hidden handles.

close all force deletes all figures, including GUIs for which CloseRequestFcn has been altered to not close the window.

status = close(...) returns 1 if the specified windows have been deleted and 0 otherwise.

4. Imread

Read image from graphics file

Syntax

A = imread(filename, fmt)

[X, map] = imread(...)

[...] = imread(filename)

[...] = imread(URL,...)

[...] = imread(...,Param1,Val1,Param2,Val2...)

Description

A = imread(filename, fmt) reads a grayscale or color image from the file specified by the string filename. If the file is not in the current folder, or in a folder on the MATLAB path, specify the full pathname.

The text string fmt specifies the format of the file by its standard file extension. For example, specify 'gif' for Graphics Interchange Format files. To see a list of supported formats, with their file extensions, use the imformats function. If imread cannot find a file named filename, it looks for a file named filename.fmt.

The return value A is an array containing the image data. If the file contains a grayscale image, A is an M-by-N array. If the file contains a true color image, A is an M-by-N-by-3 array. For TIFF files containing color images that use the CMYK color space, A is an M-by-N-by-4 array.

The class of A depends on the bits-per-sample of the image data, rounded to the next byte boundary. For example, imread returns 24-bit color data as an array of uint8 data because the sample size for each color component is 8 bits.

[X, map] = imread(...) reads the indexed image in filename into X and its associated colormap into map. Colormap values in the image file are automatically rescaled into the range [0,1].

[...] = imread(filename) attempts to infer the format of the file from its content.



[...] = imread(URL,...) reads the image from an Internet URL. The URL must include the protocol type (e.g., http://).

[...] = imread(...,Param1,Val1,Param2,Val2...) specifies parameters that control various characteristics of the operations for specific formats.

5. Title

Add title to current axes

Syntax

```
title('string')  
title(fname)  
title(...,'PropertyName',PropertyValue,...)  
title(axes_handle,...)  
h = title(...)
```

Description

Each axes graphics object can have one title. The title is located at the top and in the center of the axes.

title('string') outputs the string at the top and in the center of the current axes.

title(fname) evaluates the function that returns a string and displays the string at the top and in the center of the current axes.

title(...,'PropertyName',PropertyValue,...) specifies property name and property value pairs for the text graphics object that title creates. Do not use the 'String' text property to set the title string; the content of the title should be given by the first argument.

title(axes_handle,...) adds the title to the specified axes.

h = title(...) returns the handle to the text object used as the title.

6. Subplot

Create axes in tiled positions

Syntax

```
h = subplot(m,n,p) or subplot(mnp)  
subplot(m,n,p,'replace')  
subplot(m,n,P)  
subplot(h)  
subplot('Position',[left bottom width height])  
subplot(..., prop1, value1, prop2, value2, ...)  
h = subplot(...)  
subplot(m,n,p,'v6')
```

Description

subplot divides the current figure into rectangular panes that are numbered row wise. Each pane contains an axes object which you can manipulate using Axes Properties. Subsequent plots are output to the current pane.

h = subplot(m,n,p) or subplot(mnp) breaks the figure window into an m-by-n matrix of small axes, selects the pth axes object for the current plot, and returns the axes handle. The axes are



counted along the top row of the figure window, then the second row, etc. For example,

`subplot(2,1,1), plot(income)`

`subplot(2,1,2), plot(outgo)`

plots income on the top half of the window and outgo on the bottom half. If the Current Axes is nested in a uipanel, the panel is used as the parent for the subplot instead of the current figure. The new axes object becomes the current axes.

`subplot(m,n,p,'replace')` If the specified axes object already exists, delete it and create a new axes.

`subplot(m,n,P)`, where P is a vector, specifies an axes position that covers all the subplot positions listed in P, including those spanned by P. For example, `subplot(2,3,[2 5])` creates one axes spanning positions 2 and 5 only (because there are no intervening locations in the grid), while `subplot(2,3,[2 6])` creates one axes spanning positions 2, 3, 5, and 6.

`subplot(h)` makes the axes object with handle h current for subsequent plotting commands.

`subplot('Position',[left bottom width height])` creates an axes at the position specified by a four-element vector. left, bottom, width, and height are in normalized coordinates in the range from 0.0 to 1.0.

`subplot(..., prop1, value1, prop2, value2, ...)` sets the specified property-value pairs on the subplot axes object. Available property/value pairs are described more fully in Axes Properties. To add the subplot to a specific figure or uipanel, pass the handle as the value for the Parent property. You cannot specify both a Parent and a Position; that is, `subplot('Position', [left bottom width height], 'Parent',h)` is not a valid syntax.

`h = subplot(...)` returns the handle to the new axes object.

7. Figure

Create figure graphics object

Syntax

`figure`

`figure('PropertyName',propertyvalue,...)`

`figure(h)`

`h = figure(...)`

Description

`figure` creates figure graphics objects. Figure objects are the individual windows on the screen in which the MATLAB software displays graphical output.

`figure` creates a new figure object using default property values. This automatically becomes the current figure and raises it above all other figures on the screen until a new figure is either created or called.

`figure('PropertyName', propertyvalue,...)` creates a new figure object using the values of the properties specified. MATLAB uses default values for any properties that you do not explicitly define as arguments.

`figure(h)` does one of two things, depending on whether or not a figure with handle h exists. If h is the handle to an existing figure, `figure(h)` makes the figure identified by h the current figure, makes it visible, and raises it above all other figures on the screen. The current figure is the target for graphics output. If h is not the handle to an existing figure, but is an integer, `figure(h)` creates a figure and assigns it the handle h. `figure(h)` where h is not the handle to a figure, and is not an integer, is an error. `h = figure(...)` returns the handle to the figure object.



8. Imshow

Display image

Syntax

```
imshow(I)
imshow(I,[low high])
imshow(RGB)
imshow(BW)
imshow(X,map)
imshow(filename)
himage = imshow(...)
imshow(..., param1, val1, param2, val2,...)
```

Description

imshow(I) displays the grayscale image I.

imshow(I,[low high]) displays the grayscale image I, specifying the display range for I in [low high]. The value low (and any value less than low) displays as black; the value high (and any value greater than high) displays as white. Values in between are displayed as intermediate shades of gray, using the default number of gray levels. If you use an empty matrix ([]) for [low high], imshow uses [min(I(:)) max(I(:))]; that is, the minimum value in I is displayed as black, and the maximum value is displayed as white.

imshow(RGB) displays the truecolor image RGB.

imshow(BW) displays the binary image BW. imshow displays pixels with the value 0 (zero) as black and pixels with the value 1 as white.

imshow(X,map) displays the indexed image X with the colormap map. A color map matrix may have any number of rows, but it must have exactly 3 columns. Each row is interpreted as a color, with the first element specifying the intensity of red light, the second green, and the third blue. Color intensity can be specified on the interval 0.0 to 1.0.

imshow(filename) displays the image stored in the graphics file filename. The file must contain an image that can be read by imread or dicomread. imshow calls imread or dicomread to read the image from the file, but does not store the image data in the MATLAB workspace. If the file contains multiple images, imshow displays the first image in the file. The file must be in the current directory or on the MATLAB path.

himage = imshow(...) returns the handle to the image object created by imshow.

imshow(..., param1, val1, param2, val2,...) displays the image, specifying parameters and corresponding values that control various aspects of the image display. The following table lists all imshow parameters in alphabetical order. Parameter names can be abbreviated, and case does not matter.

Parameter	Value
'Border'	Text string that controls whether imshow includes a border around the image displayed in the figure window. Valid strings are 'tight' and 'loose'. Note: There can still be a border if the image is very small, or if there are other objects besides the image and its axes in the figure. By default, the border is set to the value returned by



Parameter	Value
	<code>iptgetpref('ImshowBorder')</code> .
'Colormap'	2-D, real, m -by-3 matrix specifying a colormap. <code>imshow</code> uses this to set the figure's colormap property. Use this parameter to view grayscale images in false color. If you specify an empty colormap (<code>[]</code>), <code>imshow</code> ignores this parameter.
'DisplayRange'	Two-element vector [LOW HIGH] that controls the display range of a grayscale image. See the <code>imshow(I,[low high])</code> syntax for more details about how to set this parameter.
'InitialMagnification'	A numeric scalar value, or the text string 'fit', that specifies the initial magnification used to display the image. When set to 100, <code>imshow</code> displays the image at 100% magnification (one screen pixel for each image pixel). When set to 'fit', <code>imshow</code> scales the entire image to fit in the window. On initial display, <code>imshow</code> always displays the entire image. If the magnification value is large enough that the image would be too big to display on the screen, <code>imshow</code> warns and displays the image at the largest magnification that fits on the screen. By default, the initial magnification parameter is set to the value returned by <code>iptgetpref('Imshow Initial Magnification')</code>. If the image is displayed in a figure with its 'WindowStyle' property set to 'docked', <code>imshow</code> warns and displays the image at the largest magnification that fits in the figure. Note: If you specify the axes position (using <code>subplot</code> or <code>axes</code>), <code>imshow</code> ignores any initial magnification you might have specified and defaults to the 'fit' behavior. When used with the 'Reduce' parameter, only 'fit' is allowed as an initial magnification.
'Parent'	Handle of an axes that specifies the parent of the image object that will be created by <code>imshow</code> .
'Reduce'	Logical value that specifies whether <code>imshow</code> subsamples the image in filename. The 'Reduce' parameter is only valid for TIFF images and you must specify a file name. Use this parameter to display overviews of very large images.
'XData'	Two-element vector that establishes a nondefault spatial coordinate system by specifying the image XData. The value can have more than two elements, but only the first and last elements are actually used.
'YData'	Two-element vector that establishes a nondefault spatial coordinate system by specifying the image YData. The value can have more than two elements, but only the first and last elements are actually used.



9. Size

Array dimensions

Syntax

`d = size(X)`

`[m,n] = size(X)`

`m = size(X,dim)`

`[d1,d2,d3,...,dn] = size(X),`

Description

`d = size(X)` returns the sizes of each dimension of array `X` in a vector `d` with `ndims(X)` elements. If `X` is a scalar, which MATLAB software regards as a 1-by-1 array, `size(X)` returns the vector `[1 1]`.

`[m,n] = size(X)` returns the size of matrix `X` in separate variables `m` and `n`.

`m = size(X,dim)` returns the size of the dimension of `X` specified by scalar `dim`.

`[d1,d2,d3,...,dn] = size(X)`, for `n > 1`, returns the sizes of the dimensions of the array `X` in the variables `d1,d2,d3,...,dn`, provided the number of output arguments `n` equals `ndims(X)`. If `n` does not equal `ndims(X)`, the following exceptions hold:

`n < ndims(X)` `di` equals the size of the *i*th dimension of `X` for `i = 1:n`, but `dn` equals the product of the sizes of the remaining dimensions of `X`, that is, dimensions `n+1` through `ndims(X)`.

`n > ndims(X)` `size` returns ones in the "extra" variables, that is, those corresponding to `ndims(X)+1` through `n`.

10. Double

Convert to double precision

Syntax

`double(x)`

Description

`double(x)` returns the double-precision value for `X`. If `X` is already a double-precision array, `double` has no effect.

11. Imagesc

Scale data and display image object

Syntax

`imagesc(C)`

`imagesc(x,y,C)`

`imagesc(...,clims)`

`imagesc('PropertyName',Property Value,...)`

`h = imagesc(...)`



Description

The `imagesc` function scales image data to the full range of the current colormap and displays the image.

`imagesc(C)` displays `C` as an image. Each element of `C` corresponds to a rectangular area in the image. The values of the elements of `C` are indices into the current colormap that determine the color of each patch.

`imagesc(x,y,C)` displays `C` as an image and specifies the bounds of the x - and y -axis with vectors `x` and `y`. If $x(1) > x(2)$ or $y(1) > y(2)$, the image is flipped left-right or up-down, respectively.

`imagesc(...,clims)` normalizes the values in `C` to the range specified by `clims` and displays `C` as an image. `clims` is a two-element vector that limits the range of data values in `C`. These values map to the full range of values in the current colormap.

`imagesc('PropertyName',PropertyValue,...)` is the low-level syntax of the `imagesc` function. It specifies only property name/property value pairs as input arguments.

`h = imagesc(...)` returns the handle for an image graphics object.

12. Addpath

Add folders to search path

Syntax

`addpath('folderName1','folderName2','folderName3' ...)`

`addpath('folderName1','folderName2','folderName3' ...position)`

`addpath folderName1 folderName2 folderName3 ... -position`

Description

`addpath ('folderName1','folderName2','folderName3' ...)` adds the specified folders to the top of the search path. Use the full path name for each folder. Use `genpath` with `addpath` to add all subfolders of folder Name. Use `addpath` statements in a `startup.m` file to modify the search path programmatically at startup.

`addpath('folderName1','folderName2','folderName3' ...position)` adds the specified folders to either the top or bottom of the search path, depending on the value of `position`.

Value of position Argument	Result
'-begin'	Add specified folders to the top of the search path.
'-end'	Add specified folders to the bottom of the search path.

`addpath folderName1 folderName2 folderName3 ... -position` is the command syntax.



13. Clf

Clear current figure window

Syntax

```
clf('reset')  
clf(fig)  
clf(fig,'reset')  
figure_handle = clf(...)
```

Description

clf deletes from the current figure all graphics objects whose handles are not hidden (i.e., their HandleVisibility property is set to on).

clf('reset') deletes from the current figure all graphics objects regardless of the setting of their HandleVisibility property and resets all figure properties except Position, Units, PaperPosition, and PaperUnits to their default values.

clf(fig) or clf(fig,'reset') clears the single figure with handle fig.

figure_handle = clf(...) returns the handle of the figure. This is useful when the figure IntegerHandle property is off because the noninteger handle becomes invalid when the reset option is used (i.e., IntegerHandle is reset to on, which is the default).

14. Imwrite

Write image to graphics file

Syntax

```
imwrite(A,filename,fmt)  
imwrite(X,map,filename,fmt)  
imwrite(...,filename)  
imwrite(...,Param1,Val1,Param2,Val2...)
```

Description

imwrite(A,filename,fmt) writes the image A to the file specified by filename in the format specified by fmt.

A can be an M-by-N (grayscale image) or M-by-N-by-3 (true color image) array, but it cannot be an empty array. For TIFF files, A can be an M-by-N-by-4 array containing color data that uses the CMYK color space. For GIF files, A can be an M-by-N-by-1-by-P array containing grayscale or indexed images — RGB images are not supported. For information about the class of the input array and the output image,

filename is a string that specifies the name of the output file.

imwrite(X, map, filename, fmt) writes the indexed image in X and its associated colormap to filename in the format specified by fmt. If X is of class uint8 or uint16, imwrite writes the actual values in the array to the file. If X is of class double, imwrite offsets the values in the array before writing, using uint8(X-1). map must be a valid MATLAB colormap. Note that most image file formats do not support colormaps with more than 256 entries.



When writing multiframe GIF images, X should be an 4-dimensional M-by-N-by-1-by-P array, where P is the number of frames to write.

`imwrite(...,filename)` writes the image to filename, inferring the format to use from the filename's extension.

`imwrite(...,Param1,Val1,Param2,Val2...)` specifies parameters that control various characteristics of the output file for HDF, JPEG, PBM, PGM, PNG, PPM, and TIFF files. For example, if you are writing a JPEG file, you can specify the quality of the output image.

15. Imfinfo

Information about graphics file

Syntax

`info = imfinfo(filename,fmt)`

`info = imfinfo(filename)`

`info = imfinfo(URL,...)`

Description

`info = imfinfo(filename,fmt)` returns a structure whose fields contain information about an image in a graphics file. filename is a string that specifies the name of the graphics file, and fmt is a string that specifies the format of the file. The file must be in the current folder or in a folder on the MATLAB path. If `imfinfo` cannot find a file named filename, it looks for a file named filename.fmt. The possible values for fmt are contained in the MATLAB file format registry.

If filename is a TIFF, HDF, ICO, GIF, or CUR file containing more than one image, info is a structure array with one element for each image in the file. For example, `info(3)` would contain information about the third image in the file.

`info = imfinfo(filename)` attempts to infer the format of the file from its contents.

`info = imfinfo(URL,...)` reads the image from the specified Internet URL. The URL must include the protocol type (e.g., `http://`)

16. Resize

Enlarge or shrink image sizes

Library

Geometric Transformations

Description

The Resize block enlarges or shrinks an image by resizing the image along one dimension (row or column). Then, it resizes the image along the other dimension (column or row).



Port	Input/Output	Supported Data Types	Complex Values Supported
Image / Input	M-by-N matrix of intensity values or an M-by-N-by-P color video signal where P is the number of color planes	• Double-precision floating point • Single-precision floating point • Fixed point • 8-, 16-, 32-bit signed integer • 8-, 16-, 32-bit unsigned integer	No
ROI	Four-element vector that defines the ROI	• Double-precision floating point (only supported if the input to the Input port is floating point) • Single-precision floating point (only supported if the input to the Input port is floating point) • 8-, 16-, 32-bit signed integer • 8-, 16-, 32-bit unsigned integer	No
Output	Resized image	Same as Input port	No
Flag	Boolean value that indicates whether the ROI is within the image bounds	Boolean	No

17. Who

List names of top-level data logging objects in Simulink data log

Syntax

```
log.who
tsarray.who
log.who('systems')
log.who('all')
```

Description

log.who or who(log) lists the names of the top-level signal logging objects contained by log, where log is the handle of a Simulink.ModelDataLogs object name.



tsarray.who or who(tsarray) lists the names of Simulink.TimeSeries objects contained by the Simulink.TsArray object named tsarray.

log.who('systems') or who(log, 'systems') lists the names of all signal logging objects contained by log except for Simulink.Timeseries objects stored in Simulink.TsArray objects contained by log.

log.who('all') or who(log, 'all') lists the names of all the Simulink.Time series objects contained by the Simulink.ModelDataLogs, Simulink. TsArray, or Simulink.SubsysDataLogs object named log.

18. Whos

List names and types of top-level data logging objects in Simulink data log

Syntax

log.whos

tsarray.whos

log.whos('systems')

log.whos('all')

Description

log.whos or whos(log) lists the names and types of the top-level signal logging objects contained by log, where log is the handle of a Simulink.ModelDataLogs object name.

tsarray.whos or whos(tsarray) lists the names and types of Simulink.TimeSeries objects contained by the Simulink.TsArray object named tsarray.

log.whos('systems') or whos(log, 'systems') lists the names and types of all signal logging objects contained by log except for Simulink.Timeseries objects stored in Simulink.TsArray objects contained by log.

log.whos('all') or whos(log, 'all') lists the names and types of all the Simulink.Timeseries objects contained by the Simulink.ModelDataLogs, Simulink.TsArray, or Simulink.SubsysDataLogs object named log.

Conclusion:



EXPERIMENT NO: 2

AIM: To study the effect of down sampling and quantization techniques on the grayscale image

Software/ Hardware:

Objective: The objective of the experiment is to,

- sampling and quantization techniques on the grayscale image.
- Display effect of down sampling and quantization with help of the MATLAB code.

Code:

```
clc;
clear all;
close all;
i=imread('cameraman.tif');
figure();
subplot(1,4,1);
imshow(i);
title('original image');
[r,c]=size(i);
a=i(1:2:r,1:2:c);
subplot(1,4,2);
imshow(a);
title('Sampling');
d=i(1:3:r,1:3:c);
subplot(1,4,3);
imshow(d);
title('Sampling');
b=i(1:4:r,1:4:c);
subplot(1,4,4);
imshow(b);
title('Sampling');
i=imread('cameraman.tif');
b=imresize(i,0.5,'nearest');
figure();
subplot(1,3,1);
imshow(b);title('nearest');

b1=imresize(i,0.5,'bilinear');
subplot(1,3,2);
imshow(b1);title('bilinear');
b2=imresize(i,0.5,'bicubic');
subplot(1,3,3);
imshow(b2);title('bicubic');
```



Simulation Results

original image



Sampling



Sampling



Sampling



nearest



bilinear



bicubic



Input:

Output:

Conclusion:.....

.....

.....



EXPERIMENT NO: 3

AIM: Write MATLAB code to enhance the visual quality of the image using point processing techniques, namely a) Image negative, b) Logarithmic transformation, c) Power law transformation and d) Gray level slicing technique.

Software/ Hardware:

Objective: The objective of the experiment is to,

- Gets basic information of the visual quality of the image using point processing techniques like a) Image negative, b) Logarithmic transformation, c) Power law transformation and d) Gray level slicing technique.
- Display all techniques on Matlab using Matlab code.

Syntax: Syntax to be covered in this experiment,

Theory:

a) Image Negative: This is an image processing technique where the colors or intensities in an image are inverted. In a grayscale image, this means that lighter areas become darker and vice versa. In a colored image, each color channel is inverted independently.

b) Logarithmic Transformation: A type of image enhancement technique where the pixel values in an image are transformed using a logarithmic function. This transformation is often used to expand the dynamic range of darker areas in the image while compressing the range of brighter areas.

c) Power Law Transformation: Also known as gamma correction, this transformation involves raising the pixel values in an image to a power law function. It is used to adjust the contrast and brightness of an image, especially in cases where the image's histogram is concentrated towards either the low or high intensity values.

d) Gray Level Slicing Technique: This technique is used to highlight specific ranges of pixel intensity values in an image. It involves thresholding the pixel values into different intensity bands or levels. Pixels within a specified range are retained, while others are set to a particular value, typically resulting in enhanced features or segmentation of objects based on their intensity levels.

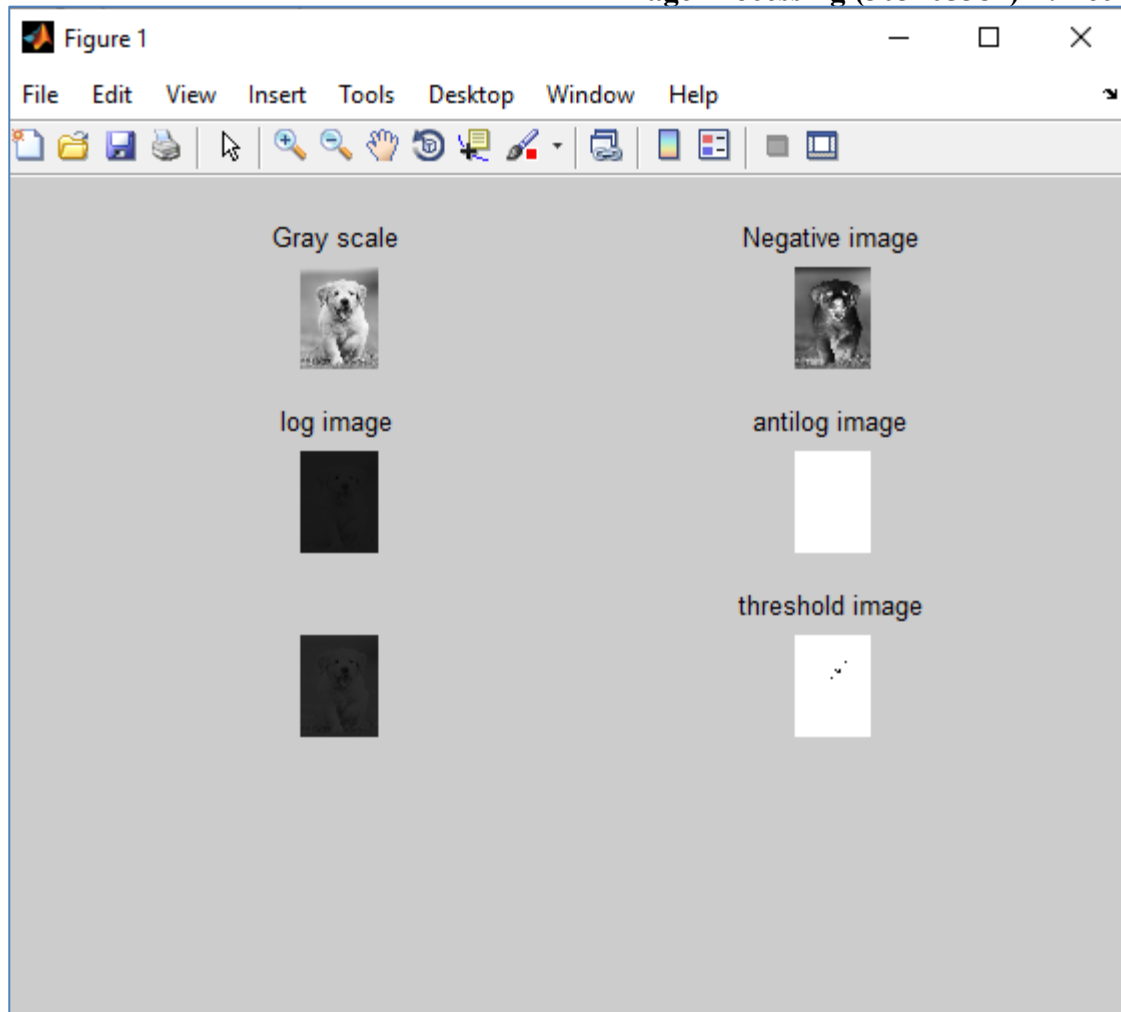
Code:

```
clc;
clear all ;
close all;
a=imread('C:\Users\Mukesh\Desktop\dog.jpg');
i=rgb2gray(a);
i=double(i);
s=255-i;
figure();
subplot(4,2,1);
```

```
imshow(uint8(i)),title('Gray scale');
subplot(4,2,2);
imshow(s,[]),title('Negative image');
c=5;
s1=c*log(1+i);
subplot(4,2,3);
imshow(uint8(s1));
title('log image');

s2=(exp(i).^(1\c))-1;
subplot(4,2,4);
imshow(uint8(s2));
title('antilog image')
gamma=0.4;
s3=c*(i.^gamma);
subplot(4,2,5);
imshow(uint8(s3));
th=15;
[r,c]=size(i);
for x=1:1:r;
    for y=1:1:c;
        if (i(x,y)<=th)
            i(x,y)=0;
        else
            i(x,y)=255;
        end
    end
end
subplot(4,2,6);
imshow(uint8(i));
title('threshold image');
```

Simulation Results:



Input: Image





Conclusion:.....

.....



EXPERIMENT NO: 4

AIM: Write MATLAB code to display the histogram of the image and enhance the visual quality of the image with the help of histogram equalization technique.

Software/ Hardware:

Objective: The objective of the experiment is to,

- Gets basic information of the histogram and also histogram equalization technique.
- Display the histogram of the image and enhance the visual quality of the image with the help of histogram equalization technique on Matlab using Matlab code.

Syntax: Syntax to be covered in this experiment,

Theory:

• Displaying the Histogram:

- The histogram of an image represents the frequency distribution of pixel intensities. It plots the number of pixels for each intensity value.
- To display the histogram, you first need to compute it. You can do this by iterating through all the pixels in the image and counting the occurrences of each intensity value.
- Once you have computed the histogram, you can plot it using a histogram plotting tool or library. The x-axis represents the intensity values, while the y-axis represents the frequency of occurrence.
- By visualizing the histogram, you can gain insights into the distribution of pixel intensities in the image. This can help you understand the overall brightness and contrast characteristics of the image.

• Histogram Equalization:

- Histogram equalization is a technique used to improve the contrast of an image by redistributing the intensity values.
- The goal of histogram equalization is to transform the intensity values of the image such that the histogram of the output image is approximately flat.
- This technique involves computing a cumulative distribution function (CDF) of the histogram and then mapping the original intensity values to new values based on this CDF.
- The mapping function is typically computed as follows: $s = T(r) = \frac{L-1}{N \times M} \sum_{j=0}^r n_j$ where $T(r)$ is the mapping function, L is the number of intensity levels (typically 256 for an 8-bit image), $N \times M$ is the total number of pixels in the image, r is the original intensity value, and n_j is the cumulative histogram up to intensity value r .
- After applying histogram equalization, the resulting image will have improved contrast and enhanced visual quality.

• Enhancing Visual Quality:

- Histogram equalization redistributes pixel intensities to utilize the entire dynamic range of intensity values available in the image.
- This can lead to better utilization of available brightness levels, resulting in improved contrast and clarity.
- However, it's important to note that histogram equalization may sometimes lead to unnatural-looking images, especially if the original histogram is heavily skewed.
- To address this, adaptive histogram equalization techniques can be used, which divide the image into smaller regions and perform histogram equalization locally. This helps to preserve local contrast while enhancing the overall image quality.

Code:

```
clc;
clear all;
close all;
i = imread('cameraman.tif');
J = imadjust(i);
figure();

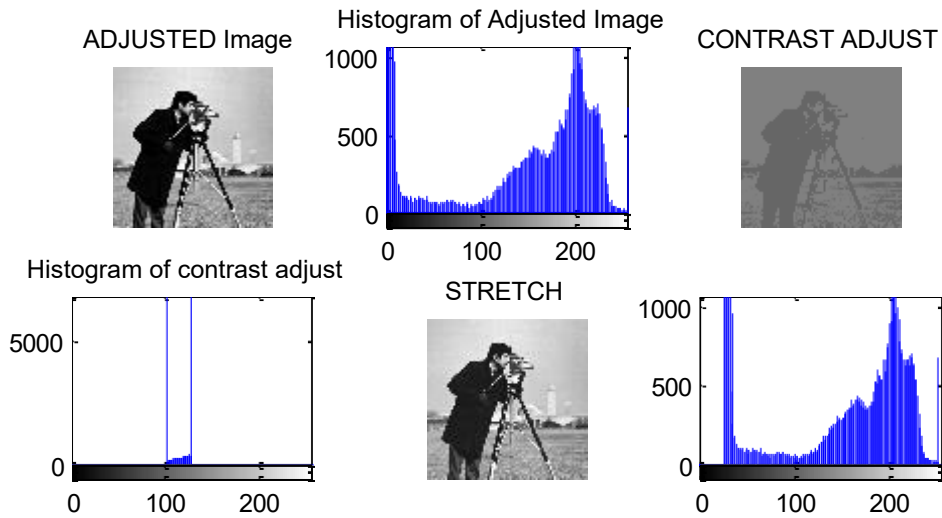
imshow(i);
title('ORIGINAL IMAGE');
figure()
subplot(3,3,1);
imshow(J);
title('ADJUSTED Image');
subplot(3,3,2);
imhist(J);
title('Histogram of Adjusted Image')
J1=imadjust(i,[0.4 0.5],[0.4 0.5]);
subplot(3,3,3);
imshow(J1);
title('CONTRAST ADJUST');
S= imadjust(i,stretchlim(i),[0.10 0.99]);
subplot(3,3,4);
imhist(J1);
title('Histogram of contrast adjust');
subplot(3,3,5);
imshow(S);
title('STRETCH');
subplot(3,3,6);
imhist(S);
title('Histogram of Stretch')
AH=adapthisteq(i);
subplot(3,3,7);
imshow(i);
title('ORIGINAL IMAGE');
subplot(3,3,8);
imshow(AH);
title('ADAPTIVE HIST');
subplot(3,3,9);
imhist(AH)
```

```
title('Histogram of Adaptive Hist')
% background = imopen(i,strel('disk',2))
% subplot(3,3,9);
% imshow(background);
% removal = i- background;
% subplot(3,3,6);
% imshow(removal);
% title('background removal');
% i3=imadjust(removal);
% figure(3);subplot(3,3,1);imshow(i);title('ORIGINAL');
% subplot(3,3,2);imshow(removal);title('BACKGROUND REMOVE');
% subplot(3,3,3);imshow(i3);title('removal adjust');
% level = graythresh(i);
% bw = im2bw(i,level);
% bw = bwareaopen(bw, 50);
% subplot(3,3,4), imshow(bw);title('binary image');
% cc = bwconncomp(bw, 4)
% cc.NumObjects
% figure(2), surf(double(background(1:8:end,1:8:end))),zlim([0 255]);
% set(gca,'ydir','reverse');
```

Simulation Results

ORIGINAL IMAGE





Conclusion:.....
.....
.....



EXPERIMENT NO: 5

AIM: Write MATLAB code to perform the bit-plane slicing method on grayscale image.

Software/ Hardware:

Objective: The objective of the experiment is to,

- Gets basic information of the bit-plane slicing on grayscale image.
- Display all techniques on Matlab using Matlab code.

Syntax: Syntax to be covered in this experiment,

Theory:

Bit-plane slicing is a technique used in image processing to decompose an image into its binary bit-planes. Each bit-plane represents a specific bit (0 or 1) of the pixel values in the image. This method is useful for various

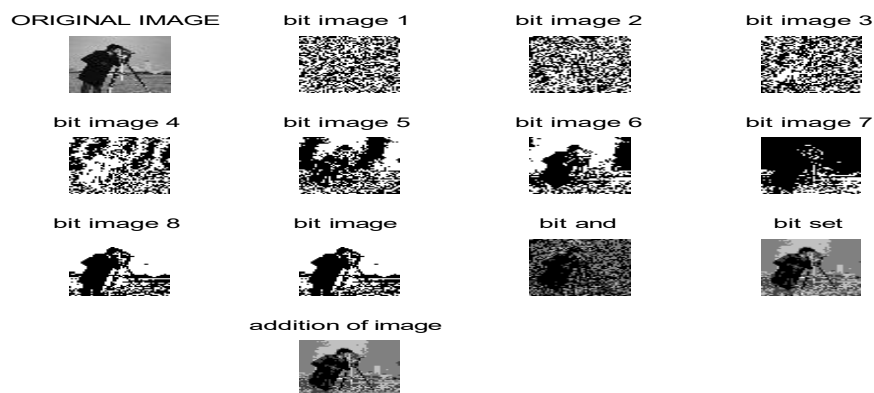
Code:

```
clc;
clear all;
close all;
i=imread('CameraMan.tif');

a1=bitget(i,1);
a2=bitget(i,2);
a3=bitget(i,3);
a4=bitget(i,4);
a5=bitget(i,5);
a6=bitget(i,6);
a7=bitget(i,7);
a8=bitget(i,8);
a9=im2bw(i);
figure()
imshow(i);
figure()
subplot(4,4,1),imshow(i),title('ORIGINAL IMAGE');
subplot(4,4,2),imshow(a1, []),title('bit image 1');
subplot(4,4,3),imshow(a2, []),title('bit image 2');
subplot(4,4,4),imshow(a3, []),title('bit image 3');
subplot(4,4,5),imshow(a4, []),title('bit image 4');
subplot(4,4,6),imshow(a5, []),title('bit image 5');
subplot(4,4,7),imshow(a6, []),title('bit image 6');
```

```
subplot(4,4,8),imshow(a7, []),title('bit image 7');
subplot(4,4,9),imshow(a8, []),title('bit image 8');
subplot(4,4,10),imshow(a9, []),title('bit image');
comb2=imlincomb(1,a5,1,a6,1,a7,1,a8);
bit1=bitand(comb2,i);
bit2=bitset(comb2,8);
subplot(4,4,11),imshow(bit1, []),title('bit and');
subplot(4,4,12),imshow(bit2, []),title('bit set');
subplot(4,4,14),imshow(comb2, []),title('addition of image');
```

Simulation Results



Conclusion:.....

.....

.....



EXPERIMENT NO: 6

AIM: Write MATLAB code to demonstrate that the convolution in spatial domain is equivalent to multiplication in the frequency domain.

Software/ Hardware:

Objective: The objective of the experiment is to,

Syntax: Syntax to be covered in this experiment,

Theory:

The statement you provided is a fundamental concept in signal processing and image processing, known as the Convolution Theorem. It states that convolution in the spatial domain (i.e., directly operating on the pixel values of an image) is equivalent to multiplication in the frequency domain (i.e., operating on the Fourier transforms of the images).

Code:

```
% Read the noisy grayscale image
noisyImage = imread('C:\Users\Mukesh\Desktop\dog.jpg');

% Convert to grayscale if the image is not already
if size(noisyImage, 3) == 3
    noisyImage = rgb2gray(noisyImage);
end

% Create a simple filter (e.g., averaging filter)
filter = fspecial('average', [3 3]);

% Convolution in the spatial domain
convSpatial = conv2(double(noisyImage), filter, 'same');

% Compute the Fourier transforms of the image and the filter
F_image = fft2(double(noisyImage));
F_filter = fft2(double(filter), size(noisyImage, 1), size(noisyImage, 2));

% Multiplication in the frequency domain
convFrequency = F_image .* F_filter;

% Compute the inverse Fourier transform to get the result in the spatial domain
convResult = ifft2(convFrequency);

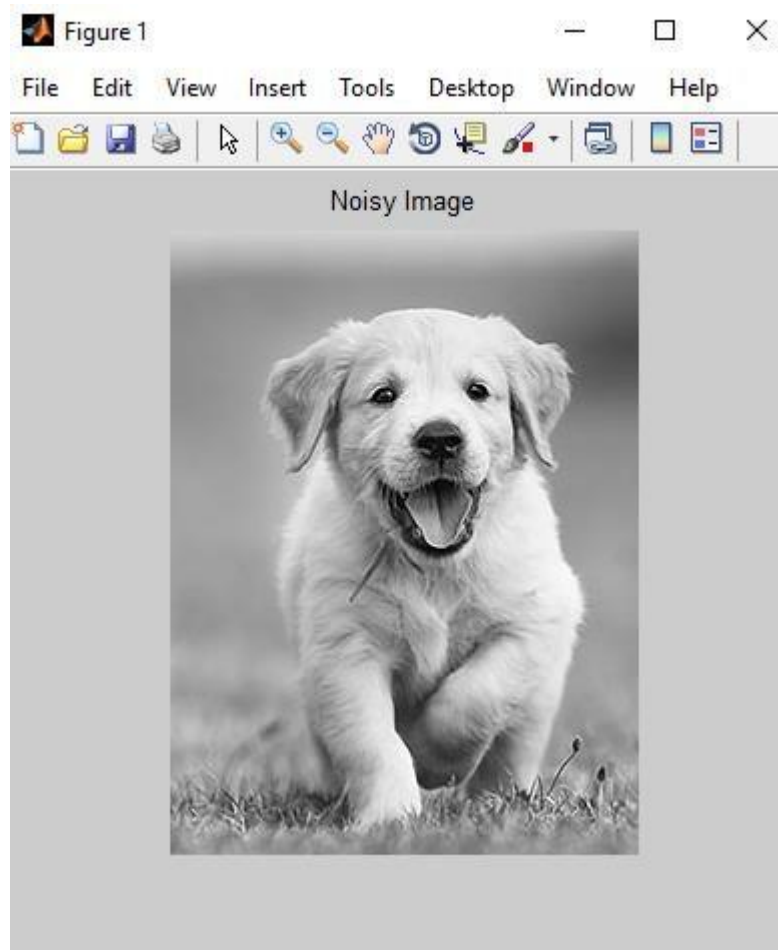
% Display the original noisy image
figure;
imshow(noisyImage);
title('Noisy Image');
```

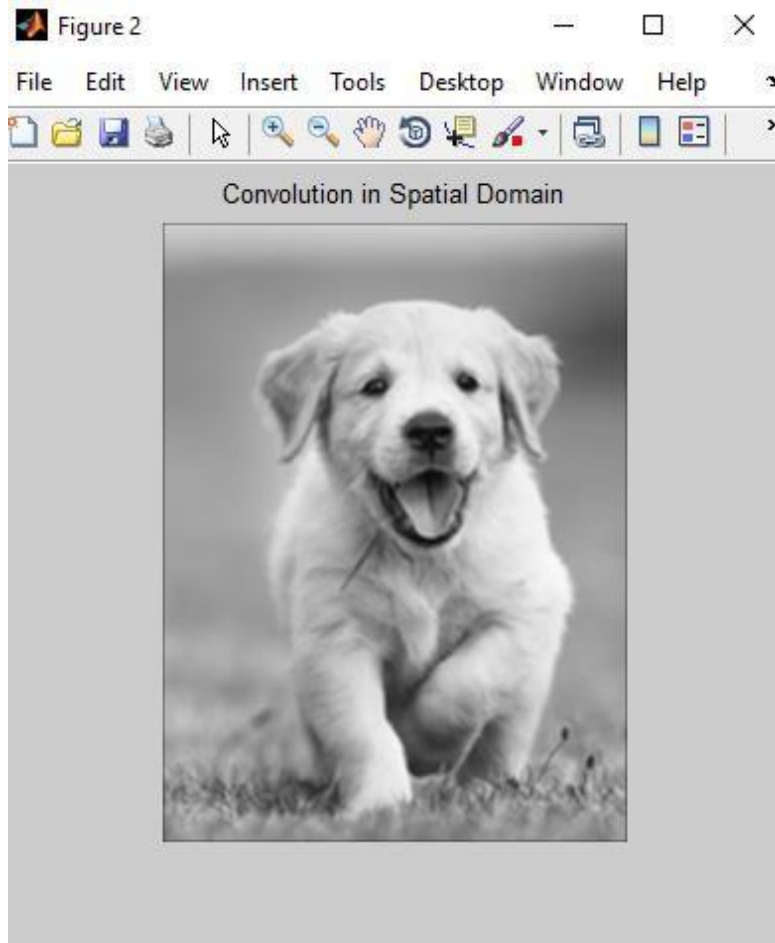
```
% Display the result of convolution in the spatial domain
figure;
imshow(uint8(convSpatial));
title('Convolution in Spatial Domain');

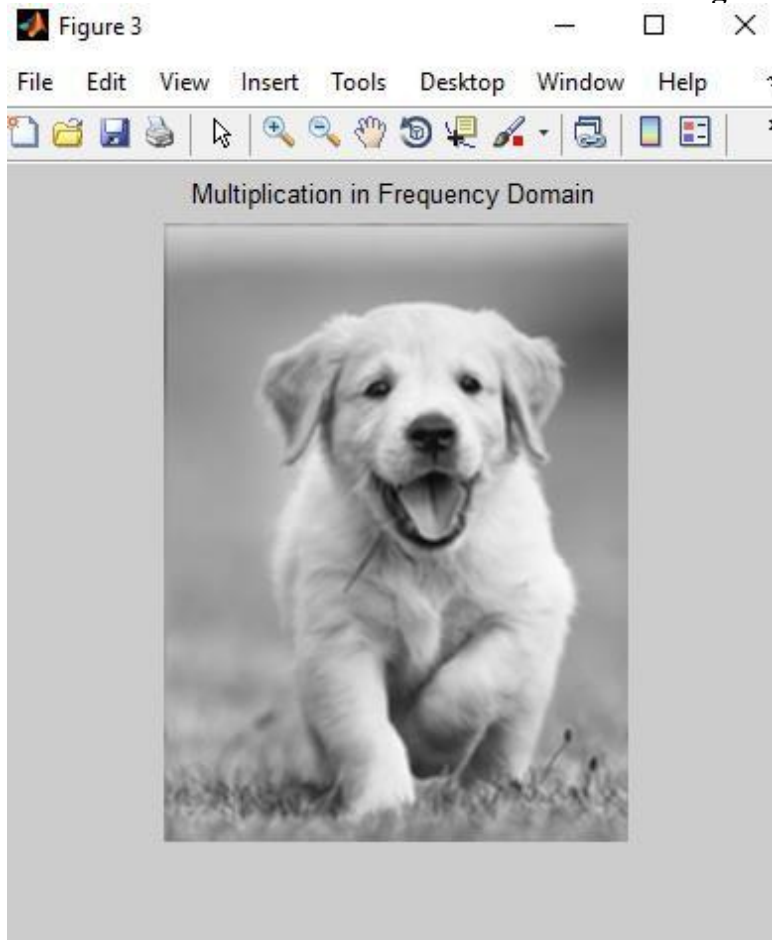
% Display the result of multiplication in the frequency domain
figure;
imshow(uint8(real(convResult)));
title('Multiplication in Frequency Domain');

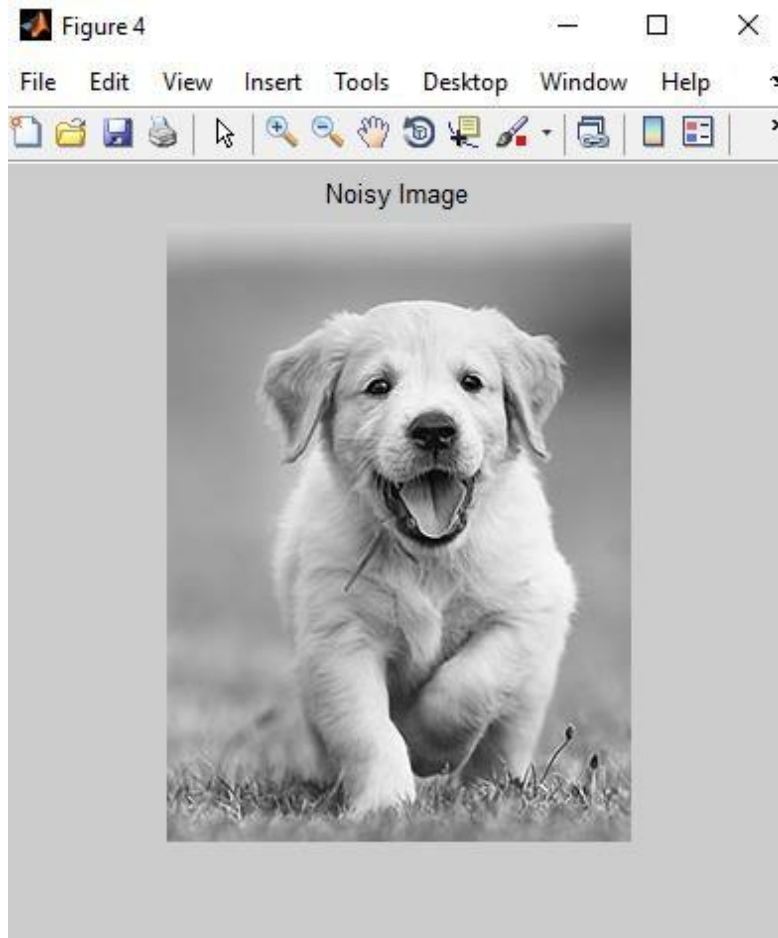
% Save the results
imwrite(uint8(convSpatial), 'C:\Users\Mukesh\Desktop\conv_spatial.png');
imwrite(uint8(real(convResult)), 'C:\Users\Mukesh\Desktop\conv_frequency.png');
```

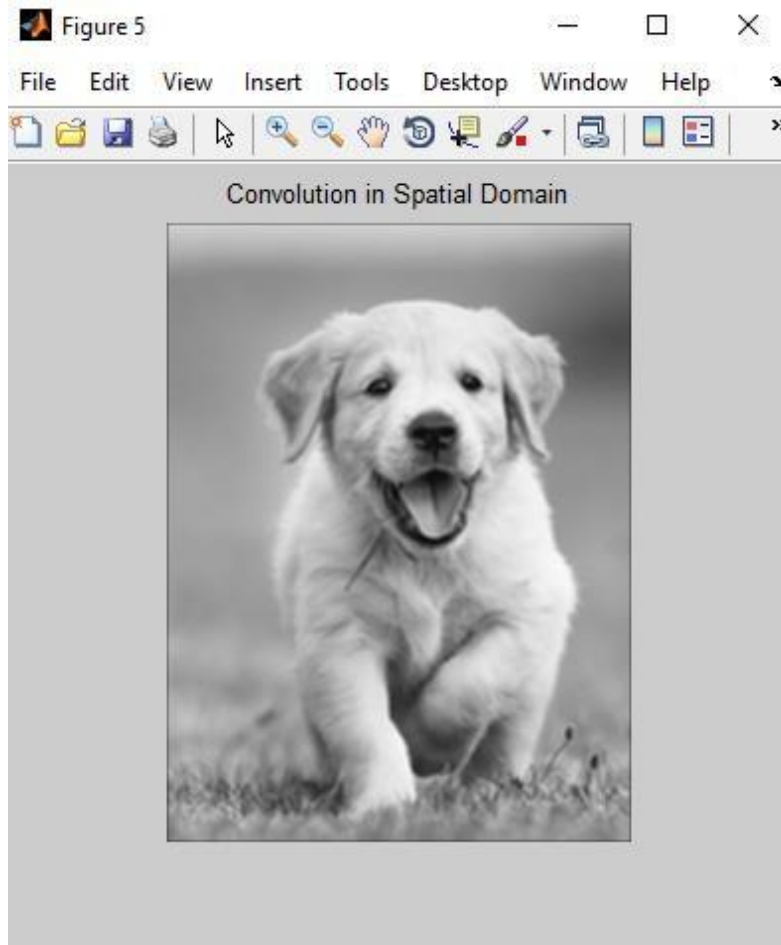
Simulation Results

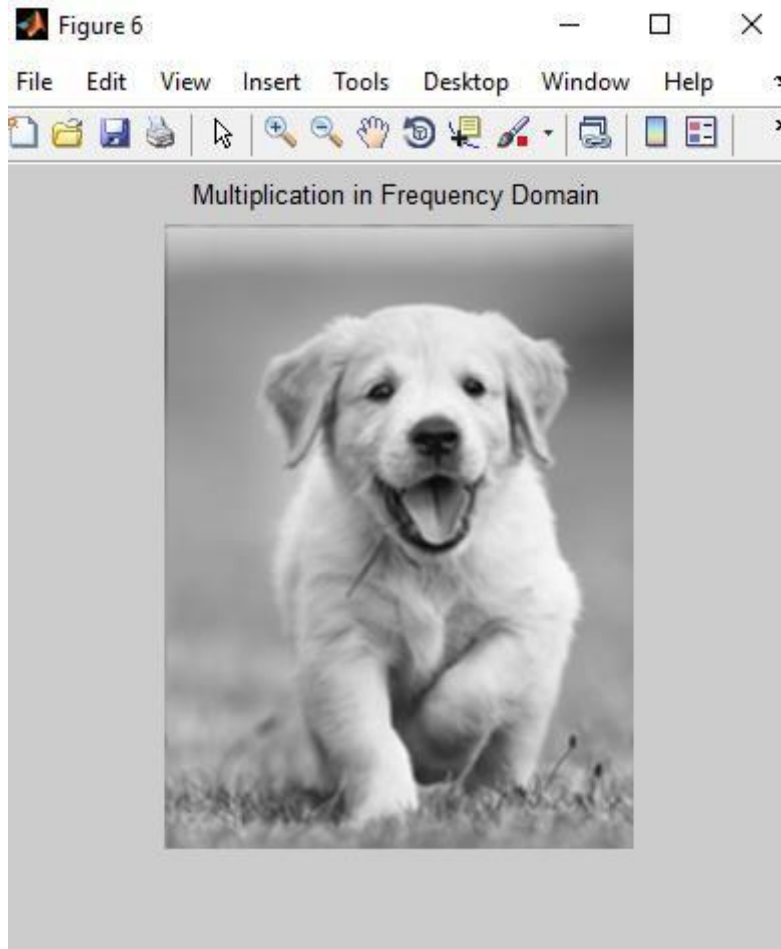












Input:

Output:

Conclusion:.....

.....



EXPERIMENT NO: 7

AIM: Write MATLAB code to restore grayscale image from noisy image with the help of image restoration techniques.

Software/ Hardware:

Objective: The objective of the experiment is to,

- Get basic information of sampling and quantization techniques on grayscale image.
- Display all techniques on MATLAB using MATLAB code.

Syntax: Syntax to be covered in this experiment,

Syntax Overview:

1. Reading an Image:

matlab
Copy code
`image = imread('filename');`

- **Purpose:** Reads an image from a file.
- **Example:** `noisyImage = imread('C:\Users\Mukesh\Desktop\dog.jpg');`

2. Converting to Grayscale:

matlab
Copy code
`grayImage = rgb2gray(image);`

- **Purpose:** Converts an RGB image to a grayscale image.
- **Example:** `noisyImage = rgb2gray(noisyImage);`

3. Displaying an Image:

matlab
Copy code
`imshow(image);`

- **Purpose:** Displays an image in a figure window.
- **Example:**

matlab
Copy code
`figure;
imshow(noisyImage);
title('Noisy Image');`

4. Applying Median Filtering:

matlab
Copy code
filteredImage = medfilt2(image);

- **Purpose:** Applies a median filter to the image to reduce noise.
- **Example:** medianFiltered = medfilt2(noisyImage);

5. Applying Wiener Filtering:

matlab
Copy code
filteredImage = wiener2(image, [m n]);

- **Purpose:** Applies Wiener filter to the image for noise reduction.
- **Parameters:**
 - image: The input image.
 - [m n]: Neighborhood size.
- **Example:** wienerFiltered = wiener2(noisyImage, [5 5]);

6. Saving an Image:

matlab
Copy code
imwrite(image, 'filename');

- **Purpose:** Writes an image to a file.
- **Example:**
`imwrite(combinedFiltered, 'C:\Users\Mukesh\Desktop\restored_image.png');`

Detailed Syntax with Examples:

1. Read the Noisy Grayscale Image:

matlab
Copy code
noisyImage = imread('C:\Users\Mukesh\Desktop\dog.jpg');

2. Convert to Grayscale if Necessary:

matlab
Copy code
if size(noisyImage, 3) == 3
 noisyImage = rgb2gray(noisyImage);
end

3. Display the Original Noisy Image:

matlab
Copy code
figure;
imshow(noisyImage);
title('Noisy Image');

4. Apply Median Filtering for Noise Reduction:

matlab
Copy code
medianFiltered = medfilt2(noisyImage);

5. Display the Median Filtered Image:

```
matlab
Copy code
figure;
imshow(medianFiltered);
title('Median Filtered Image');
```

6. Apply Wiener Filtering for Noise Reduction:

```
matlab
Copy code
wienerFiltered = wiener2(noisyImage, [5 5]);
```

7. Display the Wiener Filtered Image:

```
matlab
Copy code
figure;
imshow(wienerFiltered);
title('Wiener Filtered Image');
```

8. Combine Median and Wiener Filtering:

```
matlab
Copy code
combinedFiltered = wiener2(medianFiltered, [5 5]);
```

9. Display the Combined Filtered Image:

```
matlab
Copy code
figure;
imshow(combinedFiltered);
title('Combined Median and Wiener Filtered Image');
```

10. Save the Restored Image:

```
matlab
Copy code
imwrite(combinedFiltered, 'C:\Users\Mukesh\Desktop\restored_image.png');
```

11. Display the Restored Image:

```
matlab
Copy code
figure;
imshow('C:\Users\Mukesh\Desktop\restored_image.png');
title('Restored Image');
```

Theory:

Image restoration techniques are essential for improving the quality of images corrupted by noise. The goal is to recover the original image by reducing or eliminating the noise while preserving important details. Common techniques for restoring grayscale images from noisy images include median filtering and Wiener filtering.

Median Filtering

Median filtering is a nonlinear method that reduces noise by replacing each pixel's value with the median value of its neighboring pixels. It is particularly effective at removing salt-and-pepper noise (impulsive noise) while preserving edges and fine details.

Wiener Filtering

Wiener filtering is a linear method based on statistical principles. It aims to minimize the mean square error between the restored image and the original image. This filter adapts to the local mean and variance of the image making it effective for reducing Gaussian noise.

Combined Filtering Approach

A combined approach using both median and Wiener filtering can enhance the noise reduction process. By applying median filtering first to remove impulsive noise and then Wiener filtering to reduce Gaussian noise, a more comprehensive noise reduction can be achieved.

Implementation in MATLAB

Using MATLAB, the process of restoring a grayscale image from a noisy image involves the following steps:

1. **Reading the Noisy Image:** Import the noisy grayscale image.
2. **Median Filtering:** Apply median filtering to reduce impulsive noise.
3. **Wiener Filtering:** Apply Wiener filtering to further reduce Gaussian noise.
4. **Combining Filters:** Combine the results of median and Wiener filtering for optimal restoration.
5. **Saving and Displaying:** Save and display the restored image for analysis.

Code:

```
% Read the noisy grayscale image
noisyImage = imread('C:\Users\Mukesh\Desktop\dog.jpg');

% Convert to grayscale if the image is not already
if size(noisyImage, 3) == 3
    noisyImage = rgb2gray(noisyImage);
end

% Display the original noisy image
figure;
imshow(noisyImage);
title('Noisy Image');

% Apply median filtering for noise reduction
medianFiltered = medfilt2(noisyImage);

% Display the median filtered image
figure;
imshow(medianFiltered);
title('Median Filtered Image');

% Apply Wiener filtering for noise reduction
wienerFiltered = wiener2(noisyImage, [5 5]);

% Display the Wiener filtered image
figure;
imshow(wienerFiltered);
title('Wiener Filtered Image');

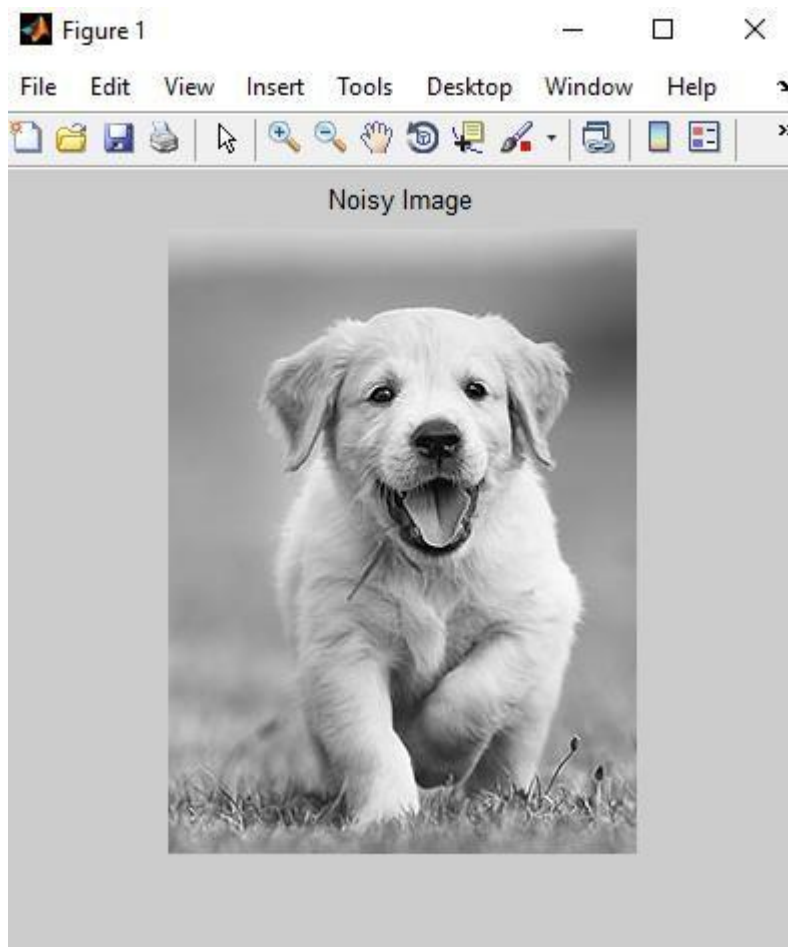
% Combine median and Wiener filtering
combinedFiltered = wiener2(medianFiltered, [5 5]);
```

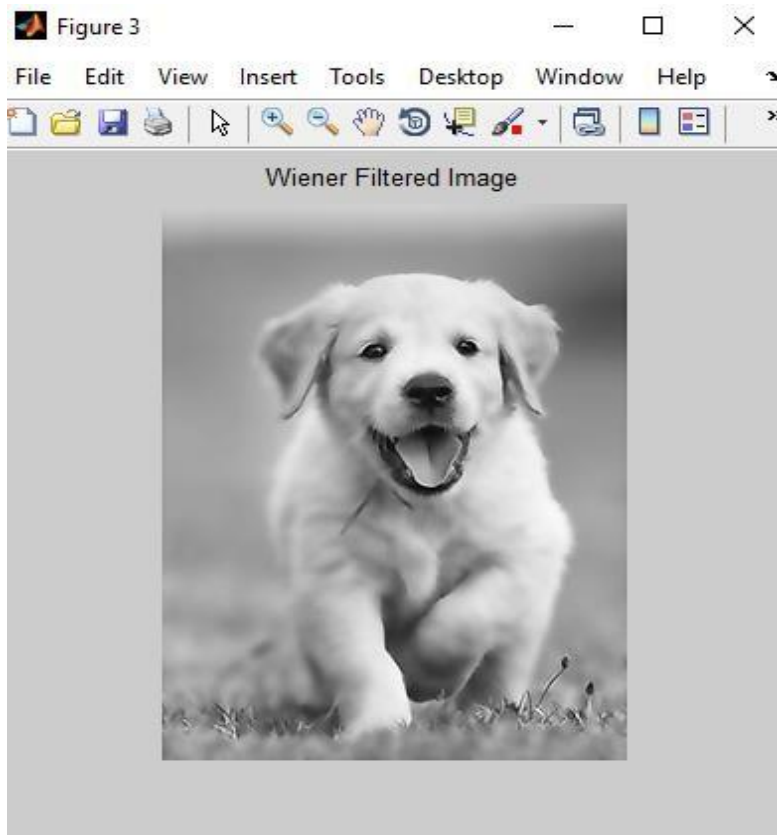
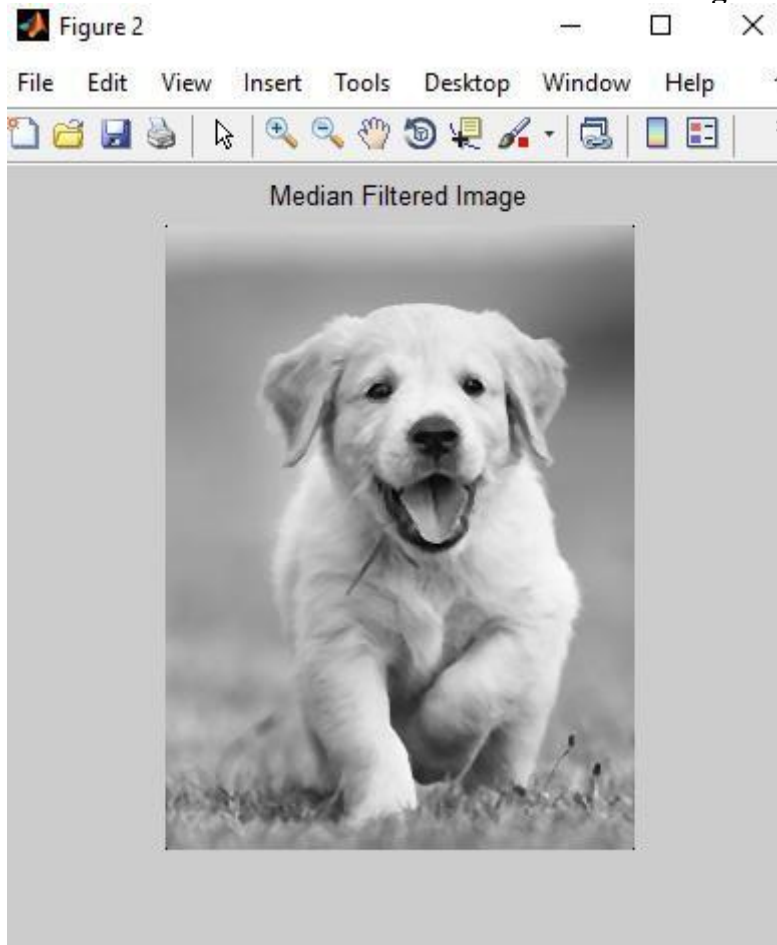
```
% Display the combined filtered image
figure;
imshow(combinedFiltered);
title('Combined Median and Wiener Filtered Image');

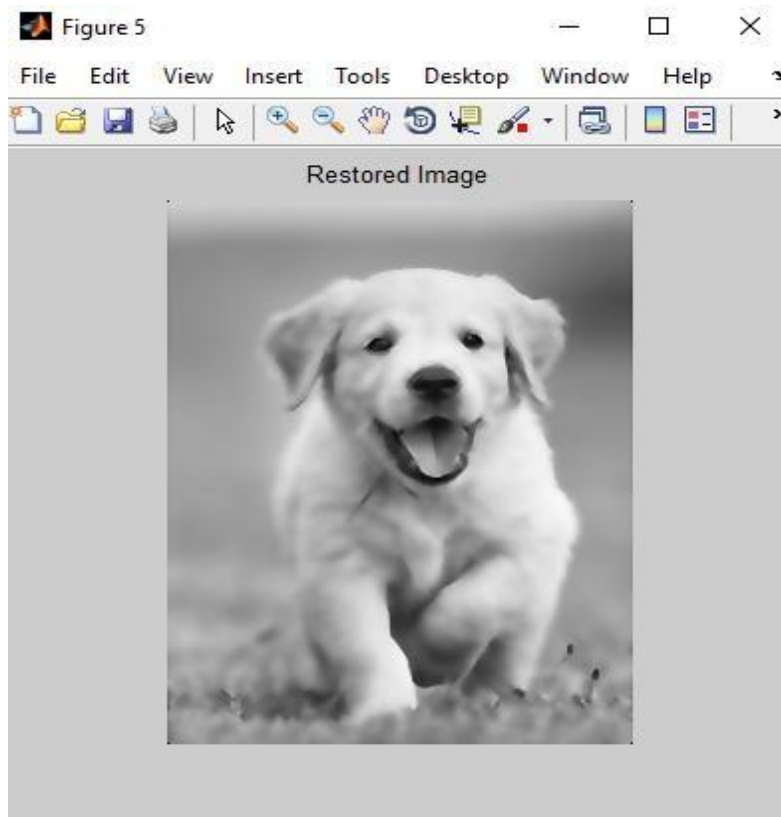
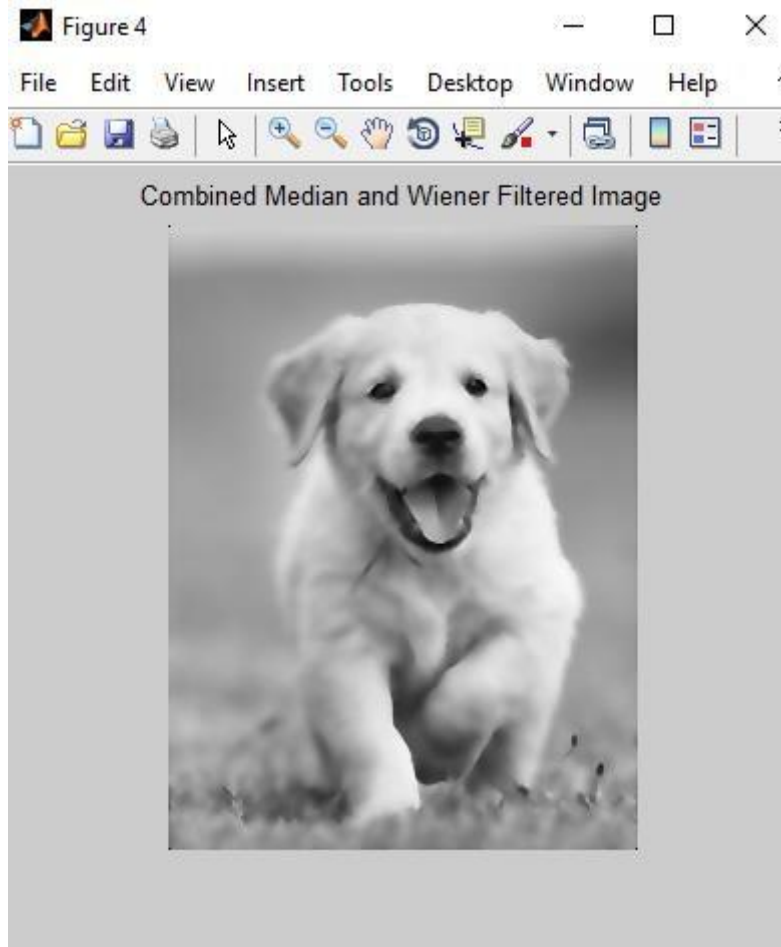
% Save the restored image
imwrite(combinedFiltered, 'C:\Users\Mukesh\Desktop\restored_image.png');

% Display the restored image
figure;
imshow('C:\Users\Mukesh\Desktop\restored_image.png');
title('Restored Image');
```

Simulation Results







Input:

Output:

Conclusion:.....

.....



EXPERIMENT NO: 8

AIM: Write MATLAB code to perform Min, Median & Max Filtering on grayscale

image.Software/ Hardware:

Objective: The objective of the experiment is to,

Syntax: Syntax to be covered in this experiment,

Theory:

- **Min Filtering:** Replaces each pixel in the image with the minimum pixel value in its neighborhood. It is often used to reduce noise in images and preserve edges.
- **Median Filtering:** Replaces each pixel with the median value in its neighborhood. It effectively removes noise while preserving edges better than mean or min filtering.
- **Max Filtering:** Replaces each pixel with the maximum pixel value in its neighborhood. It can be used for tasks like finding the brightest regions in an image.

Code:

```
clc;
clear all;
close all;

% Load the grayscale image
image_path = 'C:\Users\Mukesh\Desktop\dog.jpg';
original_image = imread(image_path);
original_image = rgb2gray(original_image); % Convert to grayscale if needed

% Define filter sizes
filter_sizes = [3, 5, 7]; % You can change these sizes as needed

% Perform Min, Median, and Max filtering for each filter size
for i = 1:length(filter_sizes)
    % Min filtering
    min_filtered_image = ordfilt2(original_image, 1, ones(filter_sizes(i)));

    % Median filtering
    median_filtered_image = medfilt2(original_image, [filter_sizes(i), filter_sizes(i)]);

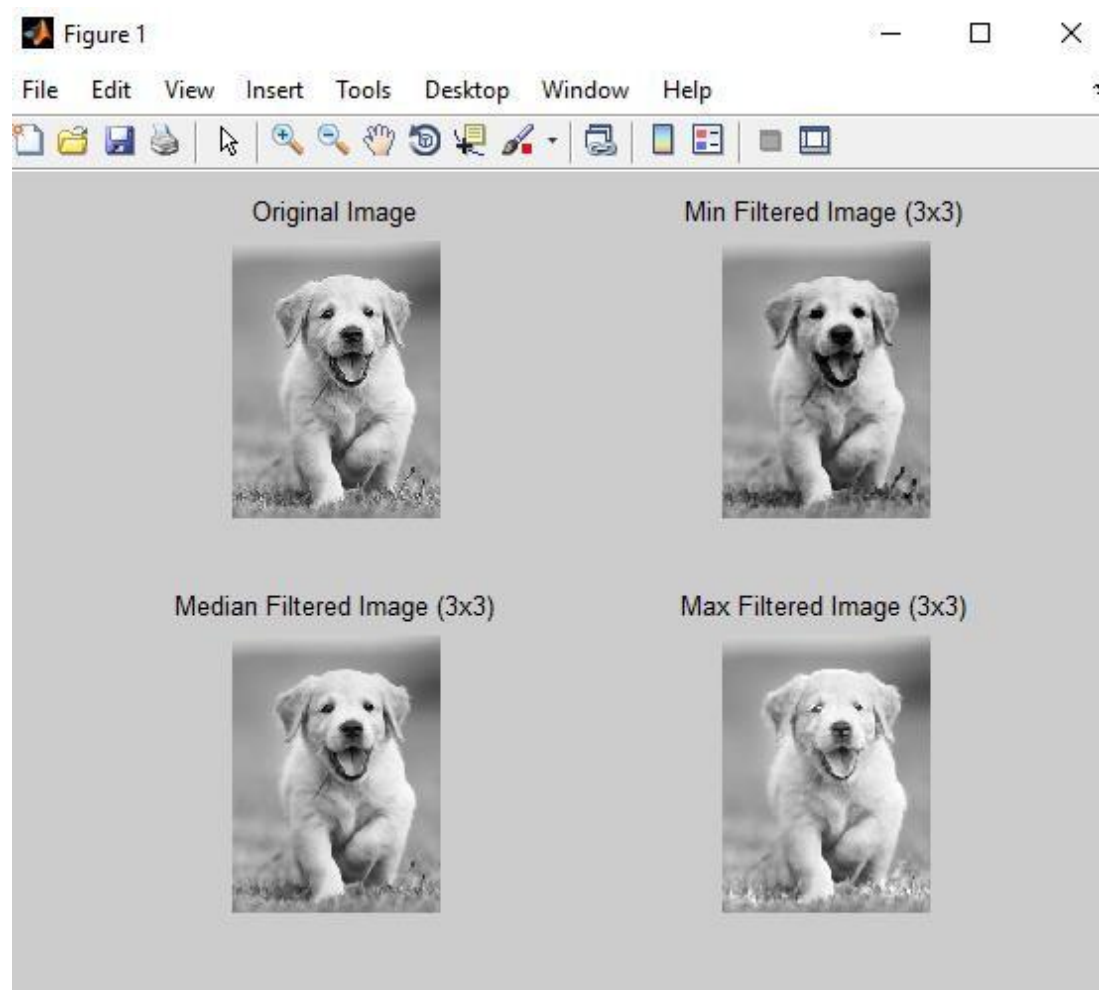
    % Max filtering
    max_filtered_image = ordfilt2(original_image, filter_sizes(i)^2, ones(filter_sizes(i)));

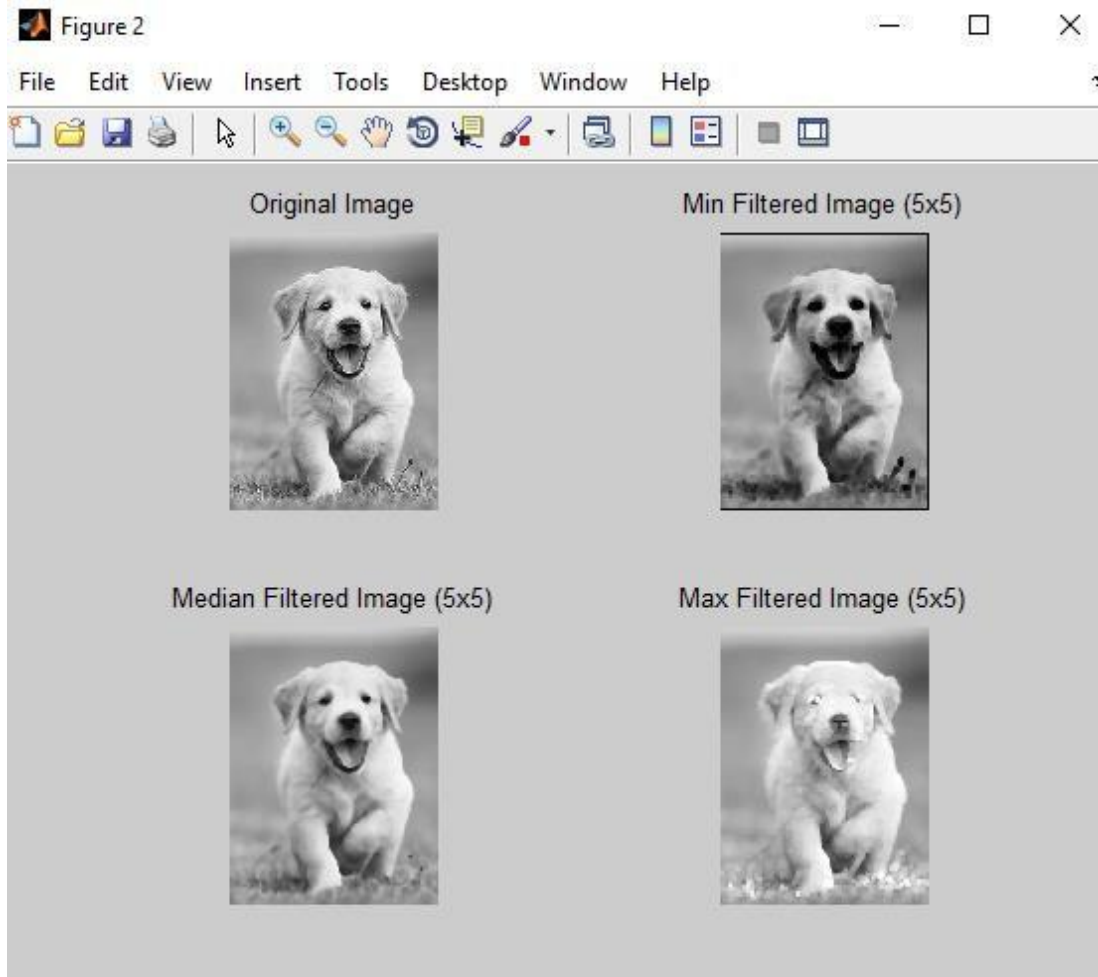
    % Display the results
```

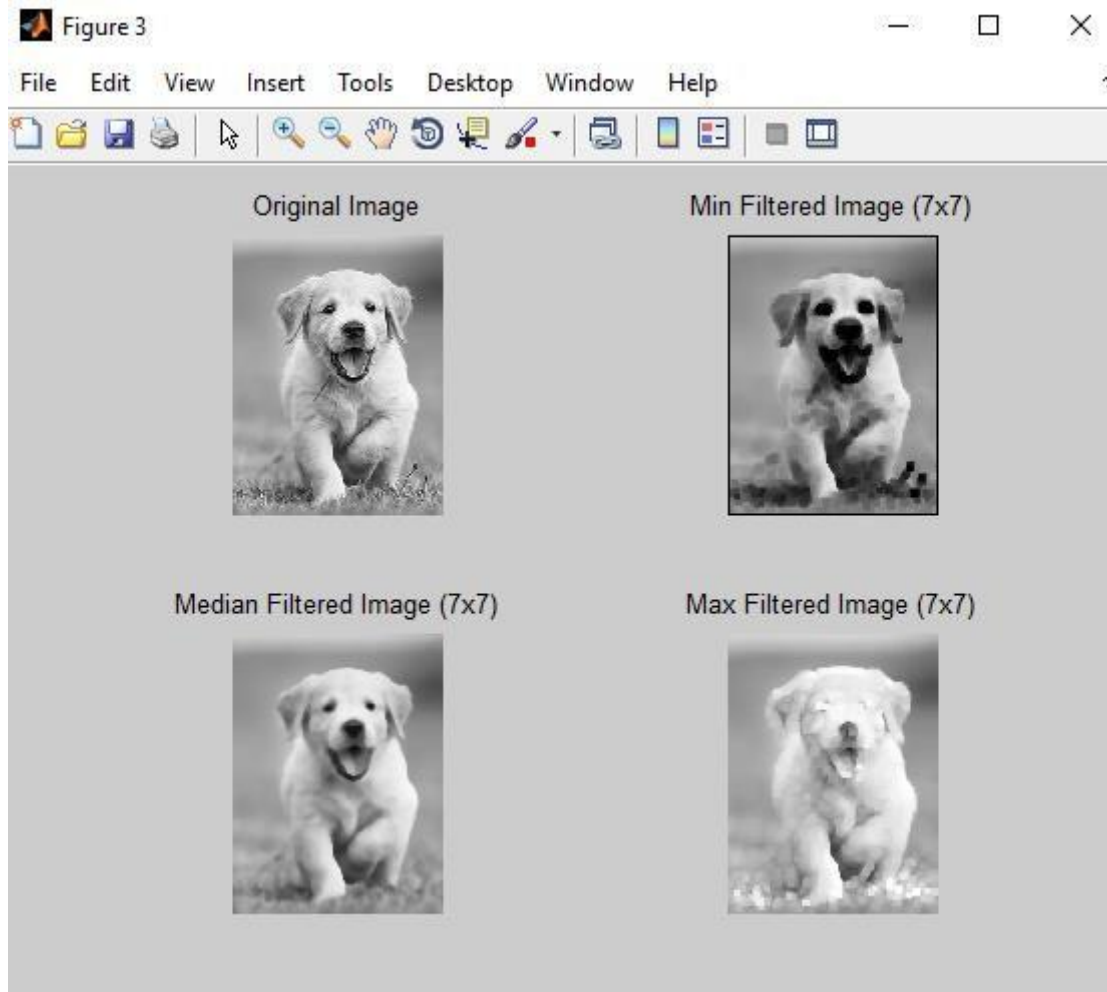


```
figure;  
subplot(2, 2, 1), imshow(original_image), title('Original Image');  
  
subplot(2, 2, 2), imshow(min_filtered_image), title(['Min Filtered Image (' , num2str(filter_sizes(i)),  
'x', num2str(filter_sizes(i)), ')']);  
  
subplot(2, 2, 3), imshow(median_filtered_image), title(['Median Filtered Image (' ,  
num2str(filter_sizes(i)), 'x', num2str(filter_sizes(i)), ')']);  
  
subplot(2, 2, 4), imshow(max_filtered_image), title(['Max Filtered Image (' , num2str(filter_sizes(i)),  
'x', num2str(filter_sizes(i)), ')']);  
  
end
```

Simulation Results







Input:

Output:

Conclusion:.....

.....



EXPERIMENT NO: 9

AIM: Write MATLAB code to blur the image using Ideal low pass, Butterworth low pass and Gaussian Low pass filter.

Software/ Hardware:

Objective: The objective of the experiment is to,

Syntax: Syntax to be covered in this experiment,

Theory:

a) **Ideal Low Pass Filter:** An ideal low pass filter is a theoretical filter that allows all frequencies below a certain cutoff frequency to pass through unchanged, while completely blocking all frequencies above that cutoff. In practice, it's impossible to create a perfect ideal filter due to issues like Gibbs phenomenon, which causes ringing artifacts in the filtered image.

b) **Butterworth Low Pass Filter:** A Butterworth low pass filter is a type of filter commonly used in signal processing and image processing. It is designed to allow frequencies below a certain cutoff frequency to pass through with minimal attenuation, while attenuating higher frequencies progressively. It provides a smoother roll-off compared to the ideal filter and is defined by its order and cutoff frequency.

c) **Gaussian Low Pass Filter:** A Gaussian low pass filter is a filter where the frequency response is defined by a Gaussian function. It attenuates higher frequencies more than lower frequencies, gradually smoothing the image. It's often used in image processing for noise reduction and blurring while preserving the overall structure of the image. The amount of smoothing is controlled by the standard deviation parameter of the Gaussian function.

Code:

```
clc;
clear all;
close all;
i=imread('C:\Users\Mukesh\Desktop\dog.jpg');
i=rgb2gray(i);
[M, N] = size(i);
FT_img = fft2(double(i));
% FT_img=fftshift(FT_img);
% Assign Cut-off Frequency
D0 = 20;
n=2;
% Designing filter
u = 0:(M-1);
idx = find(u>M/2);
u(idx) = u(idx)-M;
v = 0:(N-1);
```

```
idy = find(v>N/2);
v(idy) = v(idy)-N;
[V, U] = meshgrid(v, u);
% Calculating Euclidean Distance
D = sqrt(U.^2+V.^2);
Hil = double(D <= D0);
Hbl=1./(1 + (D./D0).^(2*n));
a=D0^2;
b=D.^2;
c=2*a;
Hgl=exp(-(D.^2)/(2*D0.^2));
Gil = Hil.*FT_img;
Gbl = Hbl.*FT_img;
Ggl = Hgl.*FT_img;

% Getting the resultant image by Inverse Fourier Transform

output_image1 = real(iff2(double(Gil)));
output_image2 = real(iff2(double(Gbl)));
output_image3 = real(iff2(double(Ggl)));

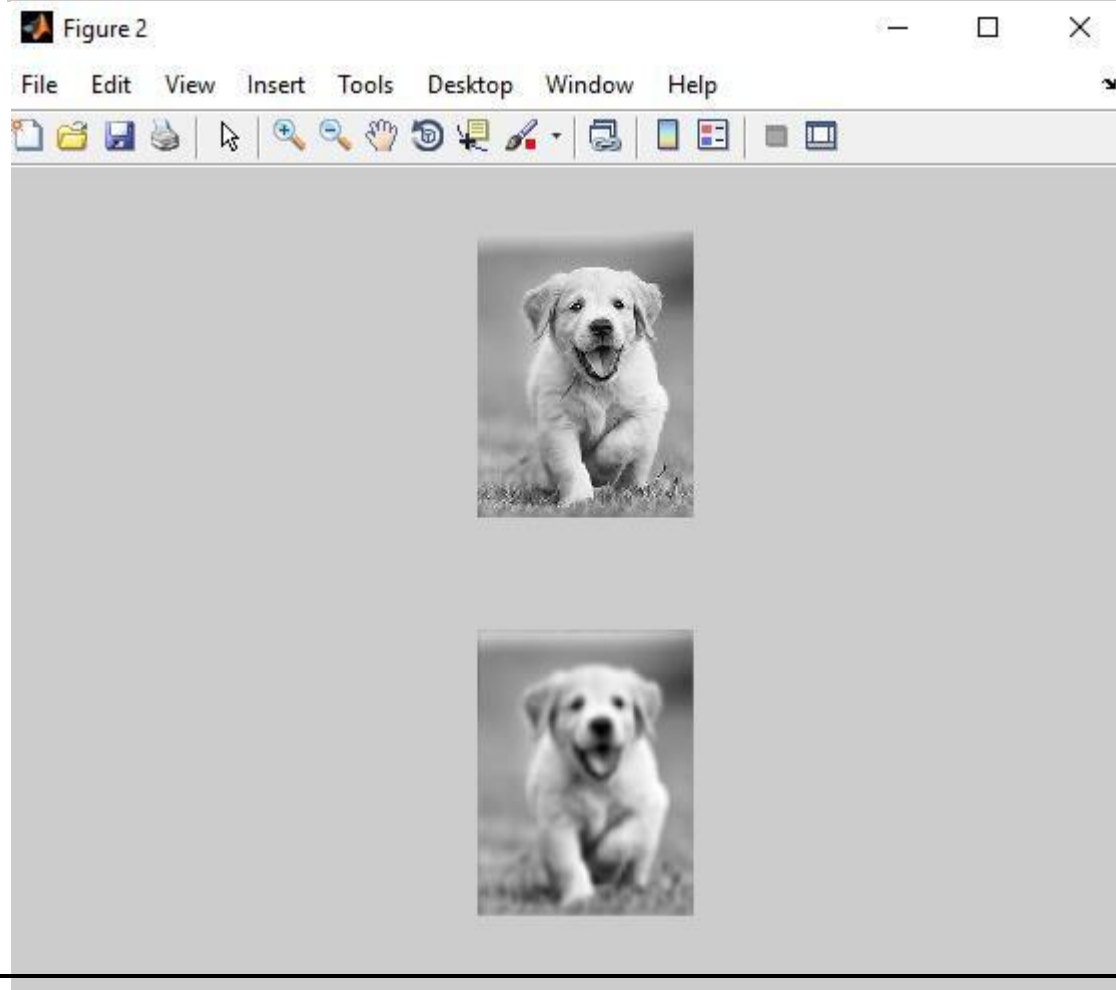
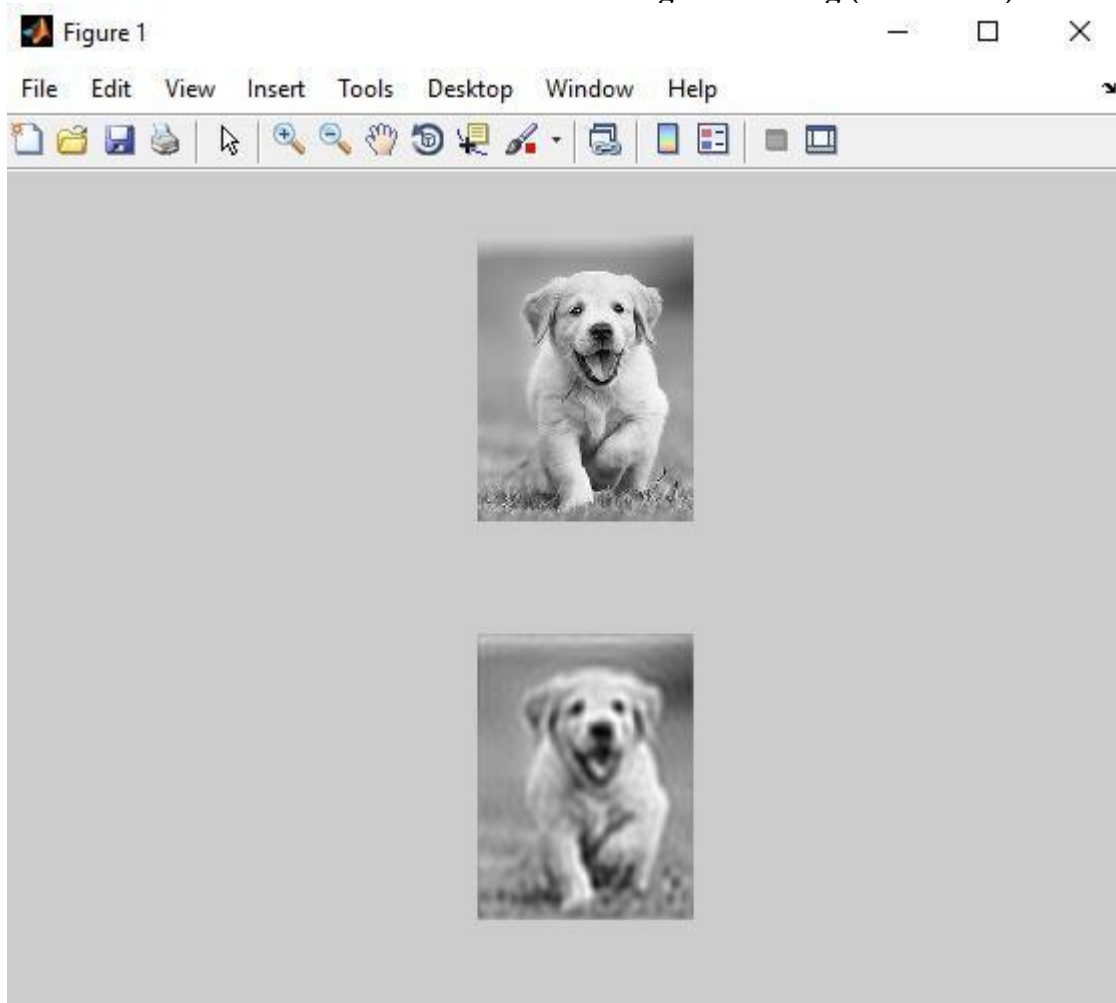
% Displaying Input Image and Output Image
figure()
subplot(2, 1, 1), imshow(i),
subplot(2, 1, 2), imshow(output_image1, [ ]);

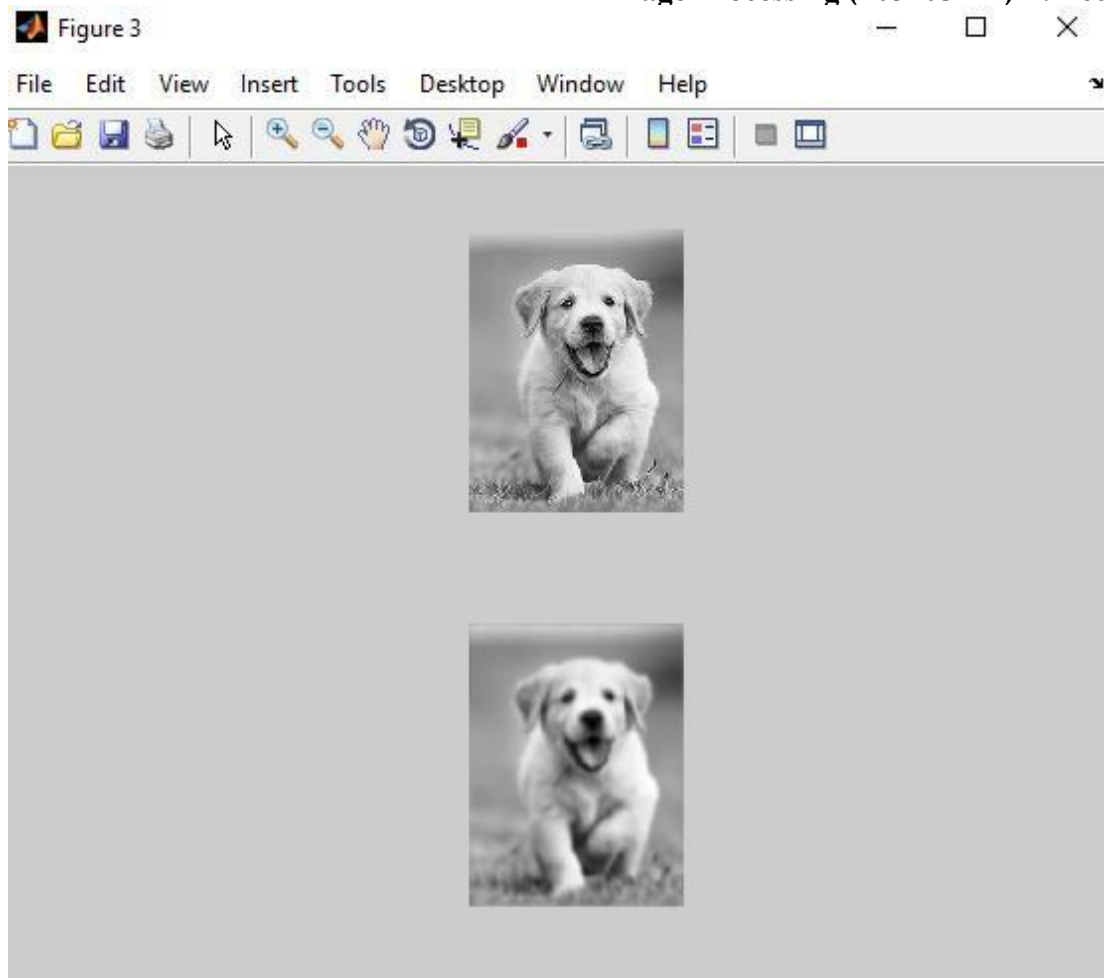
figure()
subplot(2, 1, 1), imshow(i),
subplot(2, 1, 2), imshow(output_image2, [ ]);

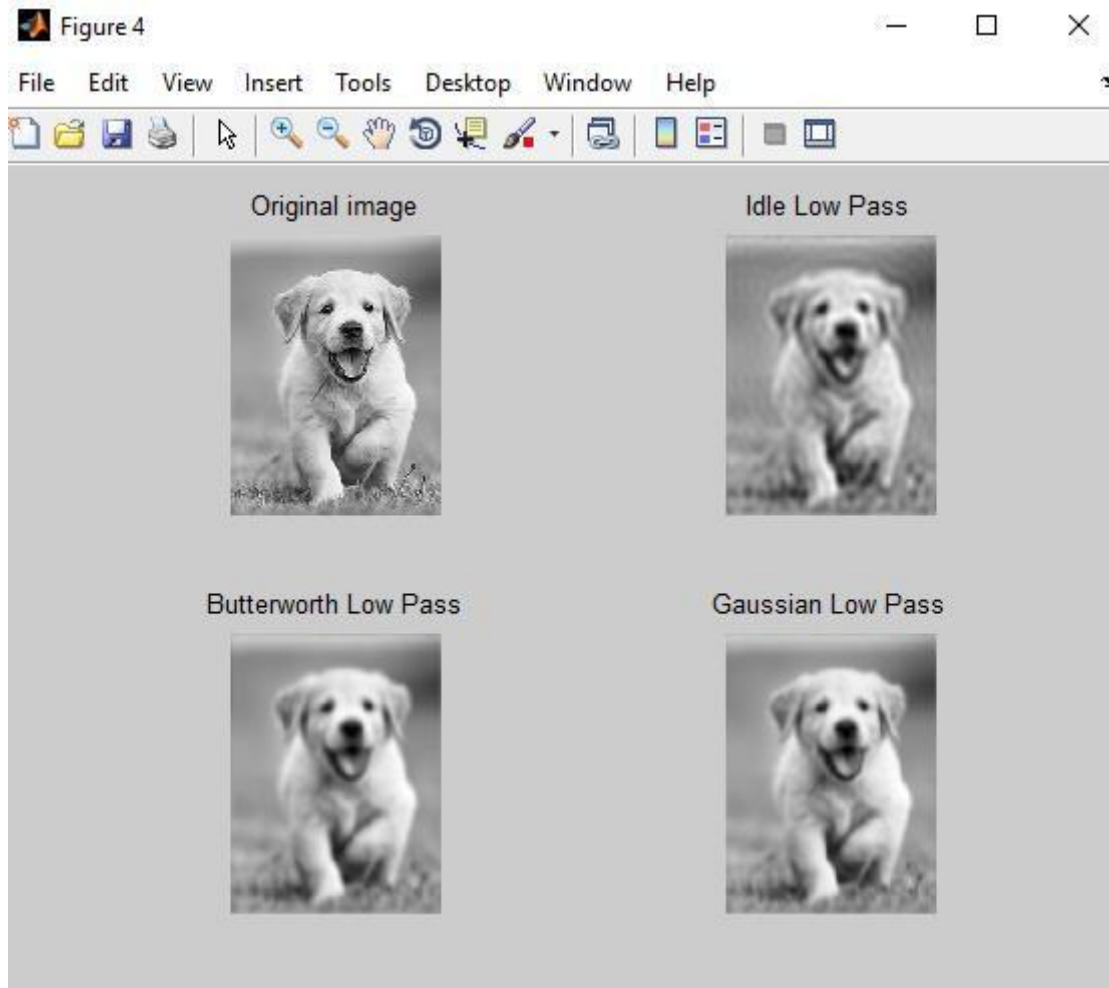
figure()
subplot(2, 1, 1), imshow(i),
subplot(2, 1, 2), imshow(output_image3, [ ]);

figure()
subplot(2,2,1);imshow(i);title('Original image');
subplot(2,2,2);imshow(output_image1, []);title('Idle Low Pass');
subplot(2,2,3);imshow(output_image2, []);title('Butterworth Low Pass');
subplot(2,2,4);imshow(output_image3, []);title('Gaussian Low Pass');
```

Simulation Results







Input:



Conclusion:.....

.....



EXPERIMENT NO: 10

AIM: Write MATLAB code to blur the image using Ideal High pass, Butterworth High pass and Gaussian High pass filter.

Software/ Hardware:

Objective: The objective of the experiment is to,

Syntax: Syntax to be covered in this experiment,

Theory:

- **Ideal High-Pass Filter:**

- The ideal high-pass filter rejects or attenuates low-frequency components (i.e., smooth areas) of an image while preserving or passing high-frequency components (i.e., edges and details).
- It is characterized by a sharp transition from attenuation to preservation at a specified cutoff frequency. Frequencies above the cutoff are passed, and frequencies below are attenuated.
- The main drawback of the ideal high-pass filter is that it has infinite support in the frequency domain, resulting in ringing artifacts in the spatial domain.

- **Butterworth High-Pass Filter:**

- The Butterworth high-pass filter is a type of high-pass filter with a smoother transition between the passband (where frequencies are preserved) and the stopband (where frequencies are attenuated) compared to the ideal filter.
- It is characterized by a parameter called the order (n), which determines the rate of rolloff from the passband to the stopband. Higher order filters have steeper rolloffs.
- Butterworth filters are preferred over ideal filters when a smoother transition is desired, as they produce fewer ringing artifacts.

- **Gaussian High-Pass Filter:**

- The Gaussian high-pass filter attenuates low-frequency components of an image while preserving high-frequency components, similar to other high-pass filters.
- It is based on the Gaussian function, which has a bell-shaped curve. In the frequency domain, the Gaussian filter reduces the weights of lower frequencies gradually, with a smooth rolloff.
- Gaussian filters are often used for noise reduction and image smoothing but can also be applied as a high-pass filter by subtracting the low-pass Gaussian filter from the original image.

Code:

```
clc;
clear all;
close all;

i = imread('C:\Users\Mukesh\Desktop\dog.jpg');
i = rgb2gray(i);
[M, N] = size(i);
FT_img = fft2(double(i));

% Assign Cut-off Frequency
D0 = 20;
n = 2;

% Designing high-pass filters
u = 0:(M-1);
idx = find(u>M/2);
u(idx) = u(idx)-M;
v = 0:(N-1);
idy = find(v>N/2);
v(idy) = v(idy)-N;
[V, U] = meshgrid(v, u);

% Calculating Euclidean Distance
D = sqrt(U.^2 + V.^2);

% Ideal high-pass filter
Hil = double(D > D0);

% Butterworth high-pass filter
Hbl = 1 ./ (1 + (D./D0).^(2*n));
Hbh = 1 - Hbl;

% Gaussian high-pass filter
Hgl = 1 - exp(-(D.^2) / (2 * D0^2));
Hgh = 1 - Hgl;

% Apply high-pass filters in frequency domain
Gih = Hil .* FT_img;
Gbh = Hbh .* FT_img;
Ggh = Hgh .* FT_img;

% Getting the resultant image by Inverse Fourier Transform
output_image1 = real(ifft2(double(Gih)));
output_image2 = real(ifft2(double(Gbh)));
output_image3 = real(ifft2(double(Ggh)));

% Displaying Input Image and Output Image
figure()
subplot(2, 1, 1), imshow(i), title('Original image');
subplot(2, 1, 2), imshow(output_image1, []), title('Ideal High Pass');

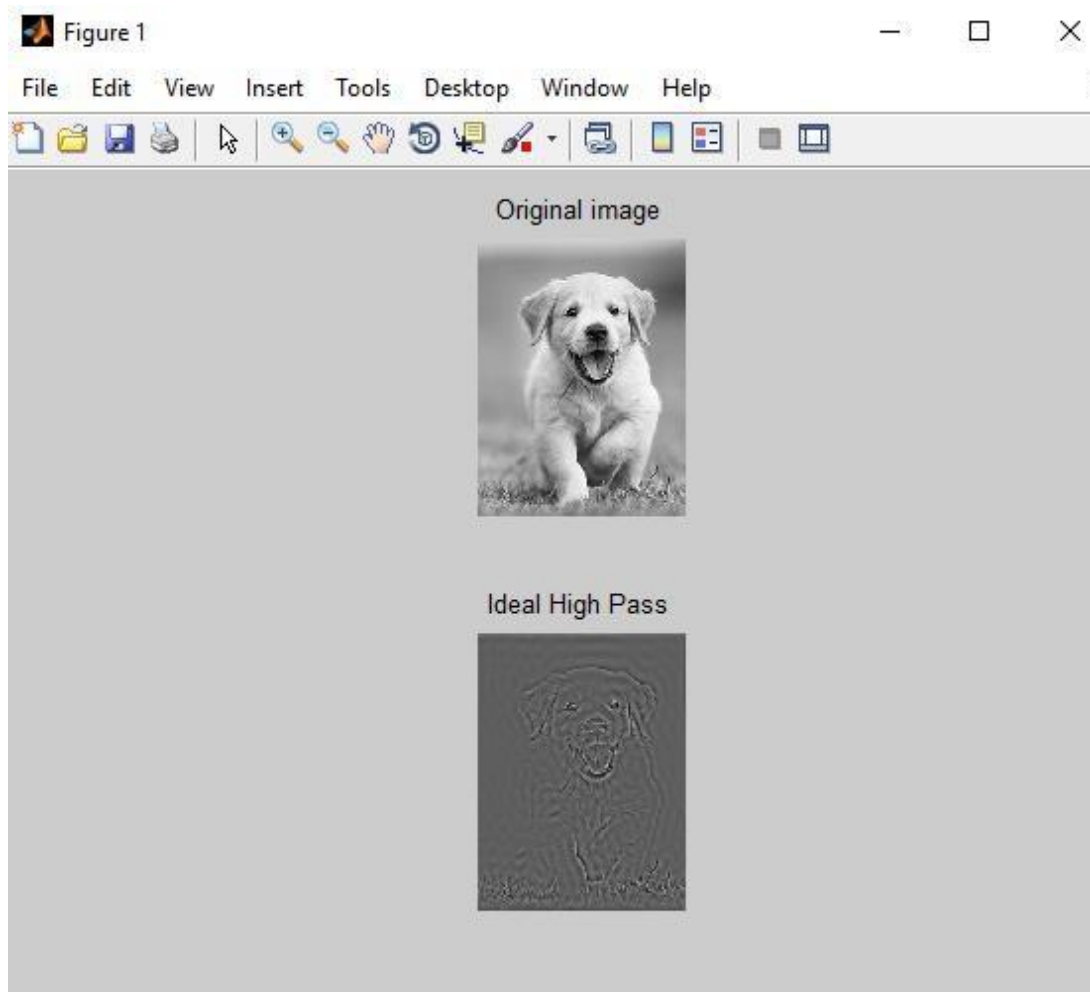
figure()
subplot(2, 1, 1), imshow(i), title('Original image');
```

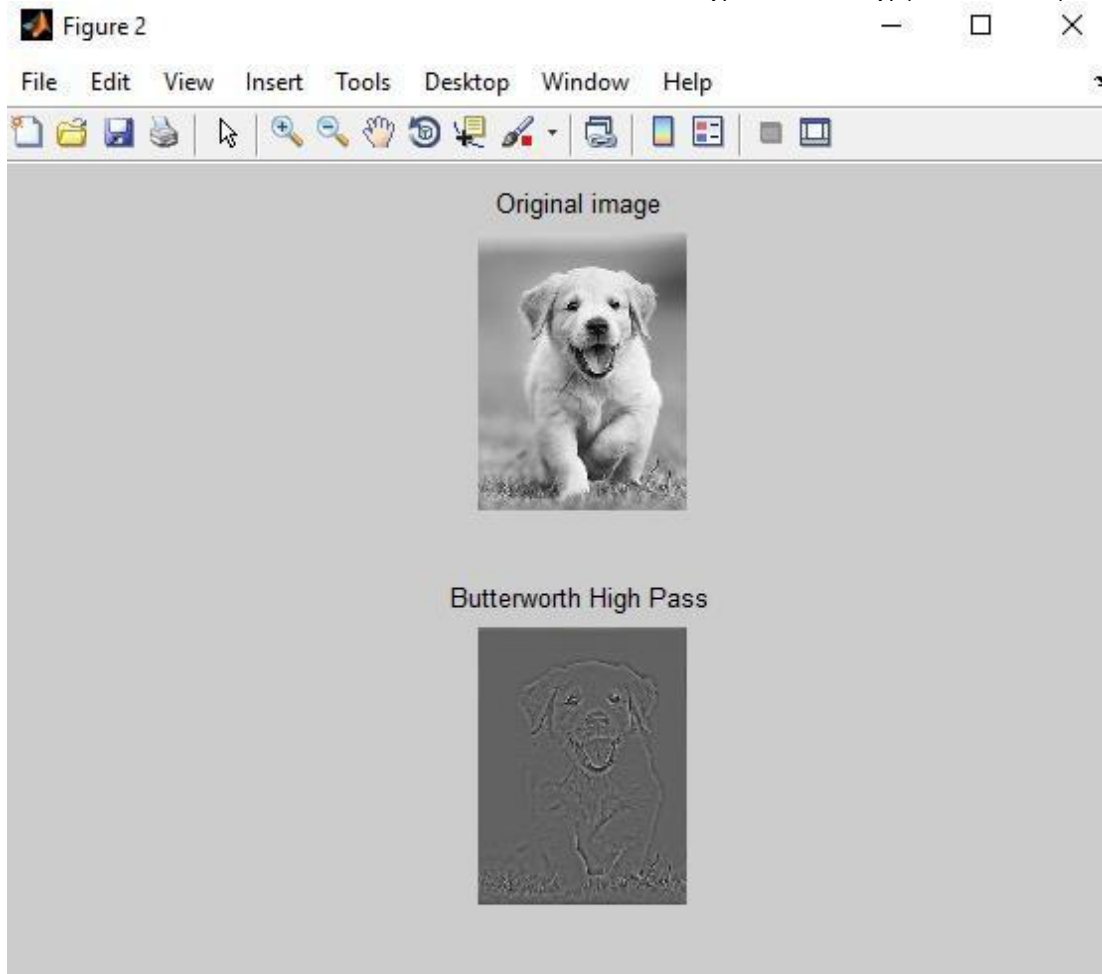
```
subplot(2, 1, 2), imshow(output_image2, []), title('Butterworth High Pass');
```

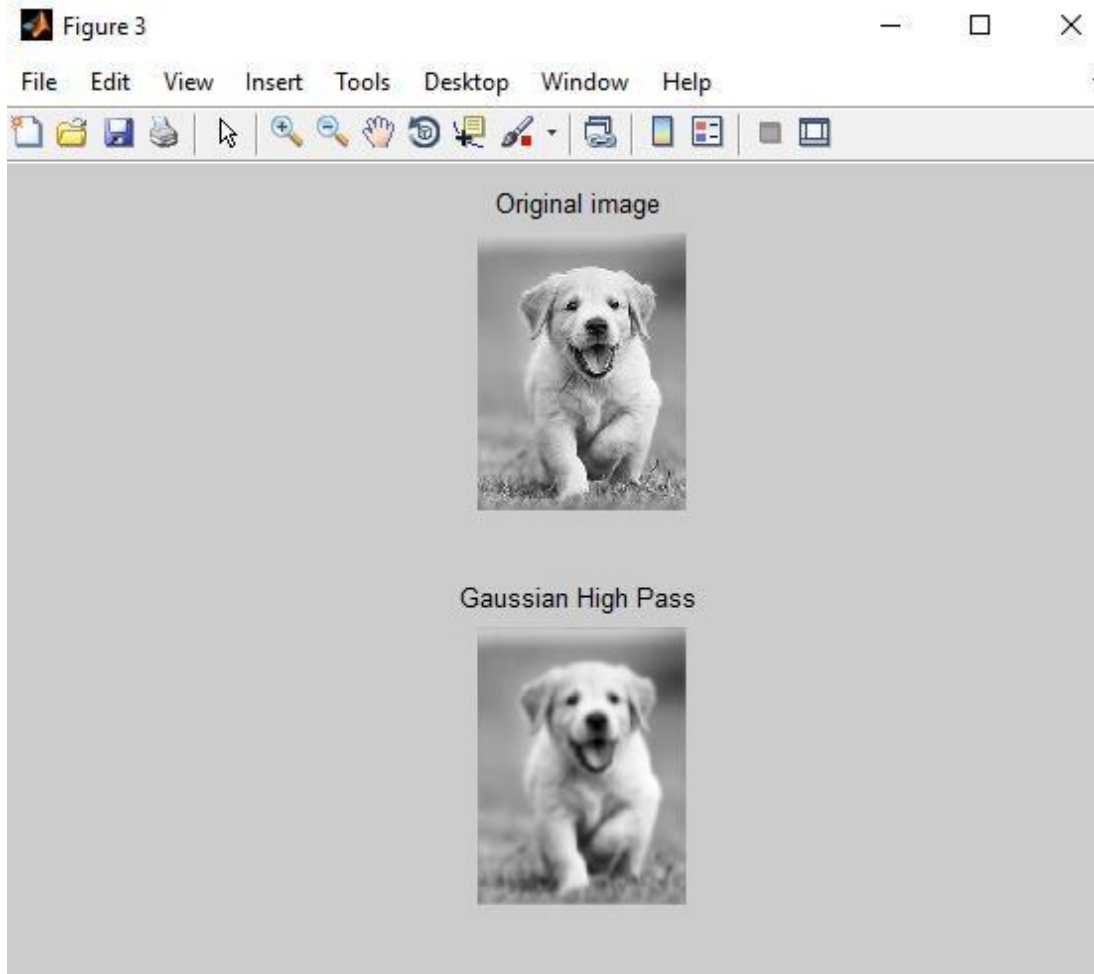
```
figure()  
subplot(2, 1, 1), imshow(i), title('Original image');  
subplot(2, 1, 2), imshow(output_image3, []), title('Gaussian High Pass');
```

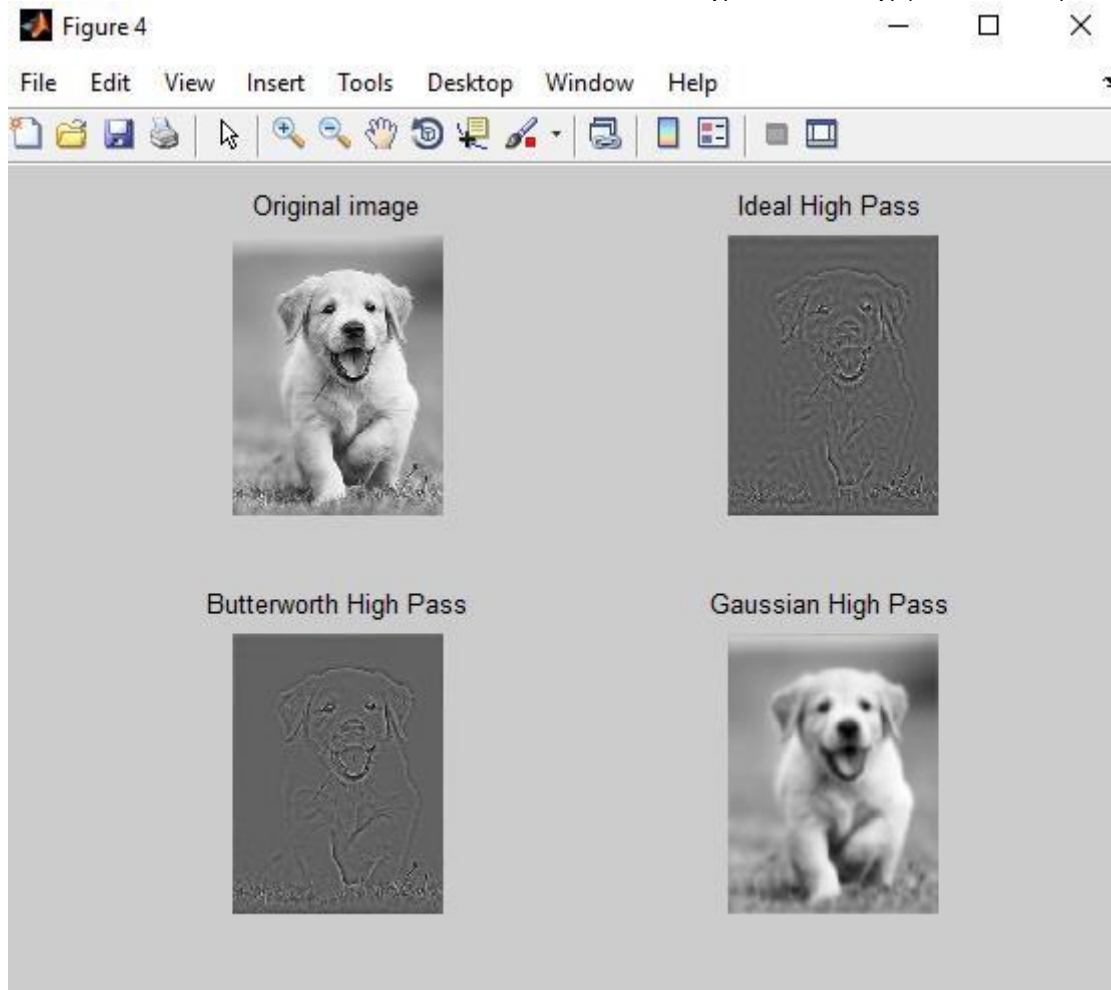
```
figure()  
subplot(2, 2, 1), imshow(i), title('Original image');  
subplot(2, 2, 2), imshow(output_image1, []), title('Ideal High Pass');  
subplot(2, 2, 3), imshow(output_image2, []), title('Butterworth High Pass');  
subplot(2, 2, 4), imshow(output_image3, []), title('Gaussian High Pass');
```

Simulation Results









Input: Image



Output:

Conclusion:.....

.....